



**DE MONTFORT
UNIVERSITY
LEICESTER**

Development Project – CTEC3451

Final Derivable

Project Title:

Implementation of a Mobile App for a Booking System



“Pc’sBooking app”

Repository: <https://github.com/Cesarvieira14/PcsBooking.git>

P2653383

César Vieira

Project Supervisor: Aboozar Taherkhani

Table of Contents

1.Acknowledgements	5
2.Abstract	6
3.Introduction	6
Background and Context	6
Problem Statement	6
Objectives	7
Scope	7
4.System Design	9
System Architecture	9
Project Structure	10
Unified Modelling Language (UML) Diagram	10
UI Design	11
5.System Major Components	12
User Major Components	12
Admin Major Components	29
6.Development Lifecycle	43
7.Critical Analysis and Reflection	46
What Went Wrong and What Went Right	46
Improvements for Future Work	46
Appraisal of the Product	46
Analysis of Approach Taken	47
8. References	48
9.Appendices	49

Table of Figures

Figure 1.MVVM Architecture Pattern (GeeksforGeeks, 2022).....	9
Figure 2. Main Activity (onCreate function).....	12
Figure 3. Welcome Page.....	12
Figure 4. CheckUser Role function	12
Figure 5. Login function.....	13
Figure 6. Login Page	13
Figure 7. Login toast missing fields.	13
Figure 8. Failed authentication toast	14
Figure 9. Firebase authentication	14
Figure 10. Forgot password pop-up.	15
Figure 11. Forgot Password function.	15
Figure 12. Forgot Password failure Toast.....	15
Figure 13. Forgot Password Success Toast.....	15
Figure 29. Time Slot Confirmation Dialog	22
Figure 30. Successful booking toast	23
Figure 31. BookViewModel bookPc function	23
Figure 32. BookViewModel fetchAvaibleDays function	24
Figure 33. AvailableDaysAdapter	25
Figure 34.PcAdapter.....	25
Figure 35.ManageFragment	26
Figure 36. Manage Bookings Page	26
Figure 37. Manage Bookings RecyclerView Item.....	26
Figure 38. Booking Adapter ViewHolder	26
Figure 39. Confirmation dialog for unbooking a reservation	27
Figure 40. Booking adapter showConfimationDialog function	27
Figure 41. Past Bookings Page.....	27
Figure 42. Past Bookings activity.....	27
Figure 43. ManageViewModel unbook function	28
Figure 44. Admin Packages	30
Figure 45.AdminBookingsFragment.....	31
Figure 46.Admin Booking Management page.....	31
Figure 47. Admin Manage bookings TabLayout.....	31
Figure 48. AdminBookingsFragment updateBookings function	31
Figure 49. Confirm Booking Cancelation toast.....	32
Figure 50. BookingDetailsActivity.....	32
Figure 51. Booking Details Page	32
Figure 52. AdminBookingsViewModel	33
Figure 53. User Management Page.....	35
Figure 54. AdminUsersFragment.....	35
Figure 55. Searching example	36
Figure 56. Search filter implementation.	36
Figure 57. User Details Page.....	36
Figure 58. User Details Activity	36
Figure 59. User Role update success toast.....	37
Figure 60. UserDetailsActivity updateUserRole function.....	37

Figure 61. UserDetailsActivity deleteUser function	37
Figure 62. AdminUsersViewModel.....	38
Figure 63. Admin machines package.....	40
Figure 64. AdminMachinesFragment.....	40
Figure 65. Admin Machine Management Page.....	40
Figure 66. Create Machine Page	41
Figure 67. CreateMachineActivity.....	41
Figure 68. Update Machine Page	41
Figure 69.UpdateMachineActivity updateMachine function.....	41
Figure 70. UpdateMachineActivity deleteMachine function	42
Figure 71. AdminMachinesViewModel	42
Figure 72. My To-do List.....	44

1. Acknowledgements

More than anything else, I want to most sincerely thank my parents. Their support, encouragement, and sacrifices have been the very cornerstone of my journey. I am just so much grateful that they had faith in my potential and saga as this provided me with the opportunity to study here in the UK. I feel deeply grateful for their ability to be there with a steady hand whenever the need arose. I would also like to express my deepest gratitude to De Montfort University for making the past three years of my life unforgettable and truly life-changing. The commitment of the university to providing a supportive and enriching environment for academics shaped my general educational experience. Equally, I am very grateful for the guidance and help received from the faculty and staff, who, in any regard, were really instrumental in my academic journey. I am greatly indebted to my project supervisor, Dr. Aboozar Taherkhani. His patience, insightful feedback, and dedicated supervision have been instrumental in the completion of my Final Year Project. His expertise and encouragement helped me in surmounting many a hurdle to achieve my goals.

I am also hugely indebted to all those involved in the module CTEC3911, Mobile Application Development. It furnished me with the theoretical background and practical experience to implement an application on Android. Expertise from instructors, together with resources, was critical in making this project successful. To colleagues and friends—there are no adequate words—that can suffice to show my gratitude. Your companionship, understanding, and support have sustained me through the ups and the downs of this journey. You listened, encouraged, and stood by me every step through this process—things which made all of it meaningful.

I want to thank my faith and God for the strengths, guidance, and hope of managing to get through all this. My faith has comforted and inspired me to remain focused and strong through difficulties. Last but certainly not least, I must thank those who have offered many different contributions during these past years to help me grow both personally and academically. This has put me in an environment where learning and development were possible and absolutely necessary for my success. Not only your impact on my academic achievements, but it also supported much of the personal growth that followed as well, for which I wish to show my gratitude sincerely.

2. Abstract

The following is the final deliverable of my Final Year Project for an Android application targeted at dynamically allocating shared resources like computers, which are generally available in large quantities to users working across different floors or open spaces (e.g., office floors or libraries) or, in more recent times, remote accessible desktops. Here, no single user owns a system, and resources are allocated on a first-come, first-served basis. The main goal behind the application is to optimise resource utilisation and increase workplace efficiency. The report starts with a basic introduction about the project idea and its motives. It further dives into the key aspects of the application and explains, in detail, the features that have been implemented. The development process includes the chosen software development lifecycle and methodology, analysed stage-by-stage, covering requirements, design, implementation, and testing phases. The report also comprises a critical review and reflection section, which evaluates the project's successes and failures and assesses the tools used. The findings in this report identify lessons learned, highlighting both successful project elements and areas needing further development for future work in similar contexts.

3. Introduction

Background and Context

The increase in the usage of technology, especially due to remote work, increases the search for innovative solutions which can manage shared environments. With the COVID-19 pandemic, remote work began to increase manifold in leaps and bounds back in 2020 (Wrycza & Maślankowski, 2020), standing at around 12.7% among full-time workers; for hybrid workers, it was around 28.2% in 2023 (Haan, 2023). This has brought about an emerging urgent need for effective management of such shared resources, including computers—many of them remotely accessed. According to a 2022 report by HP Inc., 75% of IT leaders who participated in their online survey responded that their workforce was actively using remote desktops, with 43% arguing that more than half of the working staff was actively making use of remote desktops. This naturally leads to the need for a robust resource allocation system with increased usage of remote desktops. In this paper, the solution proposed is an Android application that will efficiently allocate shared resources in terms of dynamic allocation for maximum utilization, resulting in increased efficiency at the workplace. Inspiration for doing the project also came from real-life scenarios of many organizations, educational institutions, and co-working spaces where many users want remote access to personal computers (Oluwaseun et al., 2016).

Problem Statement

The principal challenge was that, in most of the environments where multiple users needed access to computers often remotely, inefficient management of shared computing resources was the leading problem. Traditional resource allocation methods can result in underutilization or overbooking, lowering overall efficiency and user satisfaction. There needs to be a system enabled with dynamic allocation on a first-come-first-served basis; usage would get optimized, and accessibility improved.

Objectives

The project essentially aims to provide an Android application that can efficiently work out the sharing of computing resources. Following are the goals for this application:

1. Design a friendly user mobile application: Develop an Android application designed to ease the process of booking and management of shared computing resources, focusing on the seamless user experience end.
2. Resource Optimization: Design a system with the aim to do efficient allocation of the shared computers or remote desktops so that the available resources are effectively utilized and no idling of any particular resource occurs.
3. Workplace Efficiency: Save time and effort that users at present have to spend for accessing computing tools of their requirement through streamlined resource allocation.
4. Support Remote and Hybrid Work Models: Develop the application to cater to both on-site and remote users in view of the rising trend of flexible work arrangements.
5. Real-Time Availability: This will allow users to get real-time information regarding the availability of any resource, as it is in sports field reservation systems.
6. Make the System Scalable: The application should be developed in such a way that it can support users and resources spread over geographically scattered locations or departments.

Scope

As such, the scope of this project will include:

1. Requirement Analysis: This includes a detailed analysis of the needs and requirements of users and the system as a whole in terms of functional and non-functional attributes.
2. Design and Development: Implement the Android application with the right software development methodologies, following user-centred design principles.
3. User Interface Design: Developing an intuitive and accessible interface is necessary, with emphasis on applying the principles of Human-Computer Interaction that ensure usability.
4. Implementation of core functionality: User registration, resource booking, real-time availability checking, and booking management.
5. Testing and Validation: Perform thorough testing, including system acceptance tests and user acceptance tests, in order to ensure functionality and usability.
6. Documentation: Provide detailed documentation about the design of a system, including use case diagrams, sequence diagrams, and UML diagrams.
7. Deployment and User Training: The application is implemented in a real-world setting; users will be offered necessary training.
8. Performance Evaluation: Mark the effect of the application on resource usage and workplace productivity.

9. Future Recommendations: To identify possible future improvements and enhancements, based on the feedback from users' research and system performance.

4. System Design

System Architecture (MVVM)

To accomplish all of the app's objectives, a thorough analysis and selection of the architectural pattern was required. An architectural pattern determines an application's fundamental structure and organisation, including how its components interact with one another and provides a framework for both the design and implementation of software systems, ensuring a clear separation of concerns and facilitating modularity, scalability, and maintainability (GeeksforGeeks, 2024).

For this Project (Pc'sBooking app), the Model-View-ViewModel (MVVM) architecture was selected. MVVM was chosen due to the fact that is an industry-recognized software architecture pattern that overcomes all of the drawbacks of MVP and MVC design patterns by separating the data presentation logic (Views or UI) from the core business logic part of the application (GeeksforGeeks, 2022).

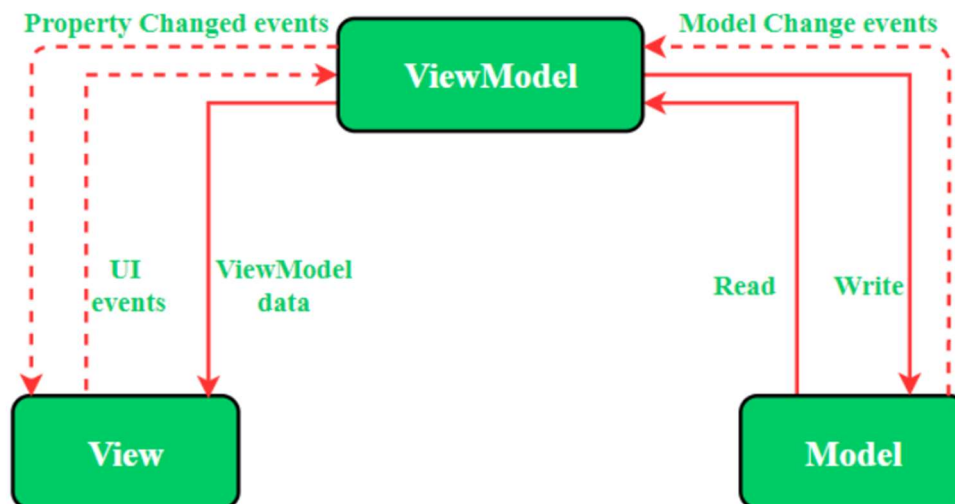


Figure 1. MVVM Architecture Pattern (GeeksforGeeks, 2022)

Code Layers of the Model-View-ViewModel architecture:

- Model --> The model encapsulates the domain and data model of the application, which often consists of business logic, intricate computations, and validation logic; model classes are typically employed in repositories that contain data access, including a set of executable domain functions, distant services, and local databases, ensuring the immutability of all app data and providing a single source of truth across many data sources (Harwani, 2023).
- View --> is responsible for creating the user interfaces displayed on the screen and consists of Android components with UI elements such as TextView, Button, or Jetpack Compose UI; these UI elements configure UI screens and send user events to the ViewModel by monitoring data or UI states from the ViewModel,

ensuring that the View contains only UI logic for displaying the screen and handling user interactions without including business logic (Harwani, 2023).

- ViewModel --> is in charge of delivering data from the model to the view (the user interface) and may also handle interactions between controls on different pages within the application; it acts as the application's brain, managing the delivery of data from the model—a collection of objects, properties, and methods—to the view, whether it's a simple text field displaying data or complex application logic on both sides(Ramotion, 2024).

Project Structure

To illustrate the organization of the project's components, a detailed project structure diagram has been provided. Due to its size, the complete image is included in **Appendix A**.

Unified Modelling Language (UML) Diagram

The UML diagram for the PC's Booking app helps us to understand better the system's architecture, including the relationships and interactions between various components. Because of its complexity, the diagram is included as a separate SVG file in the student cartridge folder. The following is a detailed explanation of the key elements, and their interactions as depicted in the UML diagram:

Model Layer:

Classes: 'Booking', 'Machines', 'Pc', 'SummaryData', 'TimeSlot', 'User'

Description: These classes represent the core data entities in the application. Each class encapsulates the relevant attributes and methods necessary for the business logic and data management. For instance, the Booking class manages attributes like bookingID, userID, machineID, and timeSlot.

View Layer (UI):

- **Components:**

Activities: MainActivity, AdminActivity, UserDetailsActivity, CreateMachineActivity, UpdateMachineActivity

Fragments: AdminBookingsFragment, AdminHomeFragment, AdminMachinesFragment, AdminProfileFragment, AdminUsersFragment,

AvailableDaysFragment, BookFragment, HomeFragment, LoginActivity, RegisterActivity, ManageFragment, ProfileFragment, TimeSlotFragment

Adapters: BookingAdapter, UsersAdapter, MachineAdapter, PcAdapter, PcListAdapter, TimeSlotAdapter

Description: The View layer is responsible for rendering the user interface and handling user interactions. Activities and Fragments form the primary UI components, displaying data and capturing user inputs. Adapters act as bridges between the data source and UI components, ensuring efficient data binding and display. For example, BookingAdapter manages the display of booking data within a list.

ViewModel Layer:

- **Classes:** AdminBookingsViewModel, AdminHomeViewModel, AdminMachinesViewModel, AdminProfileViewModel, AdminUsersViewModel, BookViewModel, HomeViewModel, LoginViewModel, ManageViewModel, ProfileViewModel
- **Description:** ViewModels act as intermediaries between the Model and View layers. They manage UI-related data and handle the business logic to provide data to the Views. This separation ensures that the View layer remains free of business logic, improving maintainability and testability. For instance, AdminBookingsViewModel handles the data operations and logic required to display and manage bookings in the admin interface.

***Accessing the Diagram**

The UML diagram is provided as SVG files for clarity and scalability. They can be viewed using any standard SVG viewer or a web browser. Please refer to the following files included in the Student Cartridge folder for detailed diagrams:

- UML.svg

UI DESIGN

[Refer to Appendix B]

5. System Major Components

5.1 User Major components

Welcome Page (MainActivity)

```
class MainActivity : AppCompatActivity() {  
  
    private lateinit var binding: ActivityMainBinding  
    private lateinit var auth: FirebaseAuth  
    private lateinit var database: DatabaseReference  
  
    // Cesar Vieira  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
  
        auth = FirebaseAuth.getInstance()  
        database = FirebaseDatabase.getInstance().reference  
  
        // Check if user is already logged in  
        val currentUser = auth.currentUser  
        if (currentUser != null) {  
            // User is signed in, navigate to the appropriate activity  
            checkUserRole(currentUser.uid)  
        } else {  
            // No user is signed in, show the main layout  
            binding.startBtn.setOnClickListener { itView.  
                navigateToLogin()  
            }  
        }  
    }  
}
```

Figure 2. Main Activity (onCreate function)

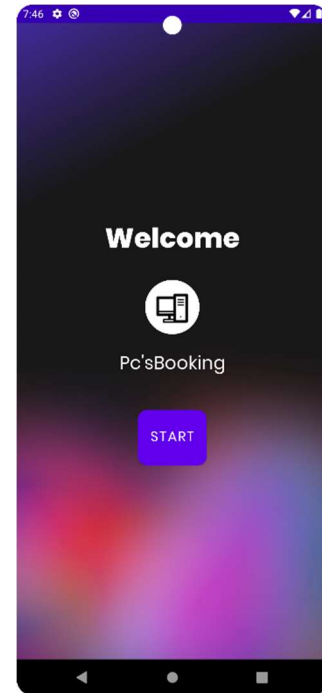


Figure 3. Welcome Page

So, the Welcome page serves as the initial interface for the app, offering a friendly greeting and the option to “Start” using the app. The welcome page (“MainActivity”) in the background manages the user navigation based on their authentication status. If a user is logged in, the activity checks the user status (normal user or admin) and directs him to the appropriate screen (normal user home or admin home). Otherwise, if a user is not authenticated, the app will redirect the user to the login screen.

```
private fun checkUserRole(userId: String) {  
    database.child(pathString: "users").child(userId).get()  
        .addOnSuccessListener { snapshot ->  
            val userData = snapshot.getValue(User::class.java)  
            if (userData != null) {  
                val intent = if (userData.admin) {  
                    Intent(packageContext, AdminActivity::class.java)  
                } else {  
                    Intent(packageContext, HomeActivity::class.java)  
                }  
                intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK  
                startActivity(intent)  
                finish()  
            } else {  
                // User data not found, navigate to login  
                showToast("User data not found. Please log in again.")  
                navigateToLogin()  
            }  
        }  
        .addOnFailureListener { exception ->  
            // Error occurred while fetching user data  
            showToast("Error fetching user data: ${exception.message}")  
            navigateToLogin()  
        }  
}
```

Figure 4. CheckUser Role function

Rationale: Checking if the user is logged in immediately upon app launch provides a seamless user experience. This approach prevents unauthenticated users from accessing restricted areas of the app and ensures that authenticated users are directed to the correct screen based on their role without unnecessary steps.

Login Page (Login Activity)

```
private fun handleLoginButtonClick() {
    val email = findViewById<EditText>(R.id.LoginEmail).text.toString()
    val password = findViewById<EditText>(R.id.LoginPassword).text.toString()
    // Check if email or password fields are empty
    if (email.isBlank() || password.isBlank()) {
        showToast("Please enter email and password")
        return
    }
    // Attempt to sign in with provided email and password
    auth.signInWithEmailAndPassword(email, password)
        .addOnCompleteListener(this) { task ->
            if (task.isSuccessful) {
                val user = auth.currentUser
                if (user != null) {
                    // Retrieve user data from the database
                    database.child("users").child(user.uid).get()
                        .addOnSuccessListener { it: DataSnapshot? } {
                            val userData = it?.getValue(User::class.java)
                            if (userData != null) {
                                // Navigate to the appropriate activity based on user role
                                if (userData.admin) {
                                    val start = Intent(packageContext, AdminActivity::class.java)
                                    startActivity(start)
                                } else {
                                    val start = Intent(packageContext, HomeActivity::class.java)
                                    startActivity(start)
                                }
                                finish() // Close the LoginActivity
                            } else {
                                // Handle case where user data is not found
                                showToast("User not found")
                            }
                        }
                } else {
                    // Handle case where user is null
                    showToast("User is null")
                }
            } else {
                // Handle login failure
                showToast("Login failed")
            }
        }
}
```

Figure 5. Login function

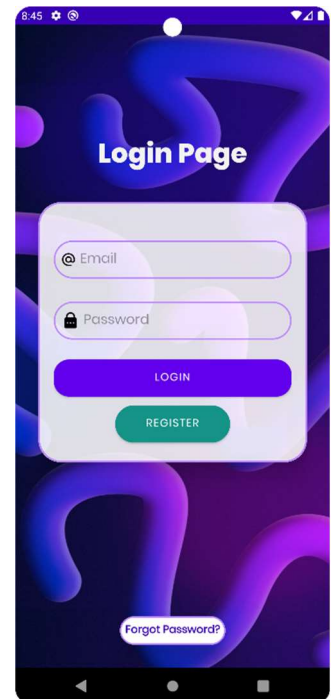


Figure 6. Login Page

The login function in LoginActivity oversees user authentication by verifying the email and password entered by the user. It uses Firebase Authentication to sign in the user and Firebase Realtime Database to retrieve additional user details for role-based navigation.

When the user clicks the login button, the app checks if the email and password fields are not blank.

Failure:

If any field is empty, it displays a Toast message prompting the user to fill in both fields.

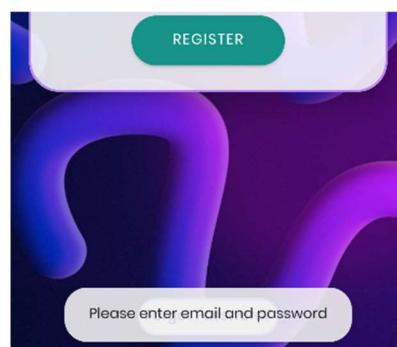


Figure 7. Login toast missing fields.

If the authentication fails or user data is not found, an appropriate Toast message is displayed to inform the user of the issue.

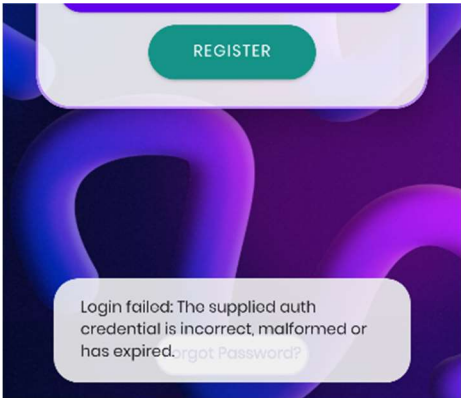


Figure 8. Failed authentication toast

Success: If the authentication is successful, the user is directed to their respective home page (normal user or admin).

Rationale: Using the Firebase Authentication, makes the handling of user credentials more secure since Firebase automatically ensures that passwords are properly hashed and stored. Also, the validation of fields prevents unnecessary server requests and also improves the user experience.

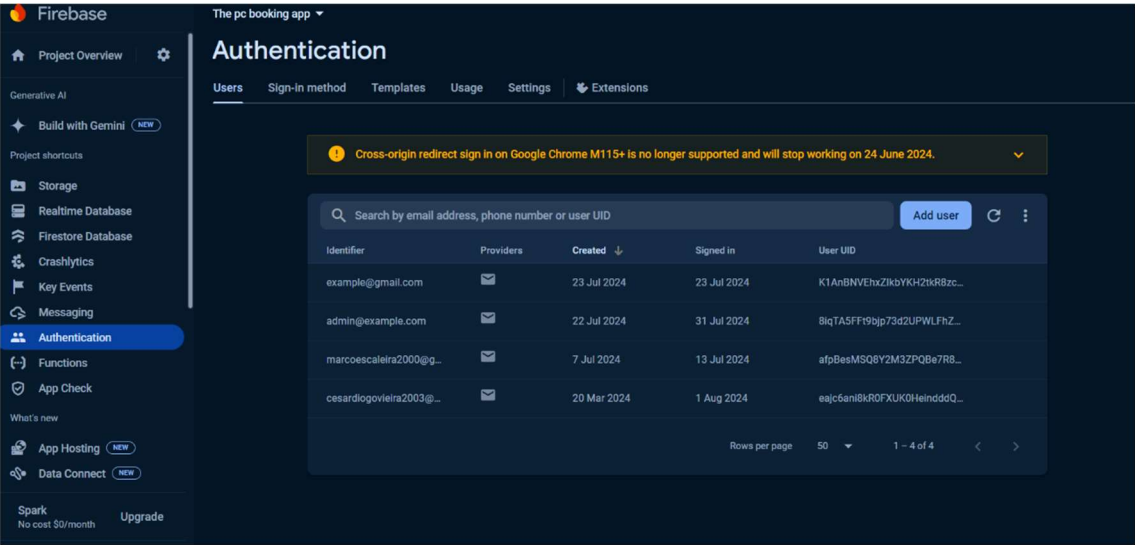


Figure 9. Firebase authentication

Users are directed to either Admin Home or normal user Home based on their role obtained from Firebase Realtime Database. This role-based navigation provides access only to features relevant to their permissions, enhancing both security and usability.

When someone enters their username and password, a Toast message informs them whether the login succeeded or failed which helps users quickly address issues like network problems or incorrect credentials, ensuring a seamless and user-friendly login experience.

Forgot Password

The function is activated by clicking the "Forgot Password" link, which opens an AlertDialog with an EditText field for the user to enter their email.

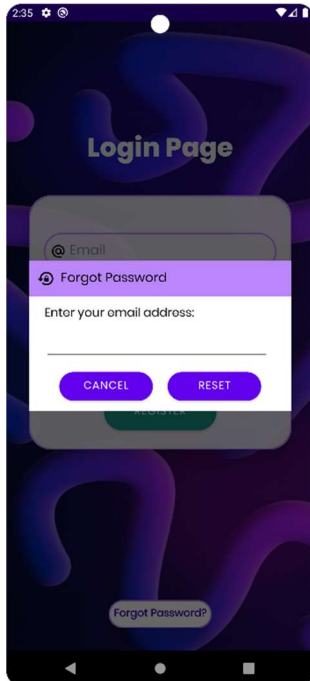


Figure 10. Forgot password pop-up.

```
private fun handleForgotPassClick() {  
    // Create and configure an AlertDialog for password reset  
    val builder = AlertDialog.Builder(context: this@LoginActivity)  
    val dialogView = layoutInflater.inflate(R.layout.popup_forgot, root: null)  
    val emailBox = dialogView.findViewById<EditText>(R.id.emailBox)  
    builder.setView(dialogView)  
    val dialog = builder.create()  
  
    // Set up click listener for the reset button in the dialog  
    dialogView.findViewById<Button>(R.id.btnReset).setOnClickListener { it View!  
        val userEmail = emailBox.text.toString()  
        // Validate email input  
        if (userEmail.isEmpty() || !Patterns.EMAIL_ADDRESS.matcher(userEmail).matches()) {  
            Toast.makeText(  
                context: this@LoginActivity,  
                text: "Enter your registered email ",  
                Toast.LENGTH_SHORT  
            ).show()  
            return@setOnClickListener  
        }  
        // Send password reset email  
        auth.sendPasswordResetEmail(userEmail).addOnCompleteListener { task ->  
            if (task.isSuccessful) {  
                Toast.makeText(context: this@LoginActivity, text: "Check your email", Toast.LENGTH_SHORT)  
                    .show()  
                dialog.dismiss() // Close the dialog on success  
            } else {  
                Toast.makeText(context: this@LoginActivity, text: "Unable to send, failed", Toast.LENGTH_SHORT)  
                    .show()  
            }  
        }  
    }  
}
```

Figure 11. Forgot Password function.

The email entered is then checked to ensure it is not empty and conforms to standard email formatting using `Patterns.EMAIL_ADDRESS`. Depending on the success or failure of the operation, the user receives a Toast message indicating the outcome, such as a successful email dispatch or an error message if the operation fails.

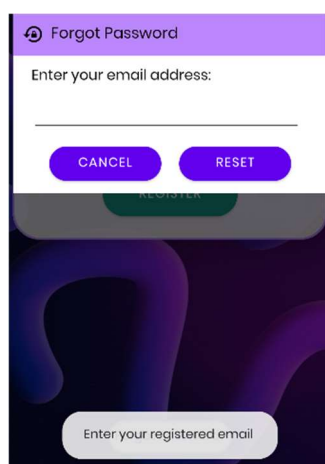


Figure 12. Forgot Password failure Toast.

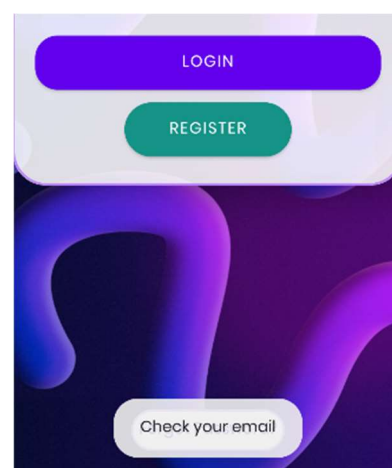


Figure 13. Forgot Password Success Toast

Sending the Reset Email: Upon validation, Firebase Authentication sends a password reset email to the provided address.

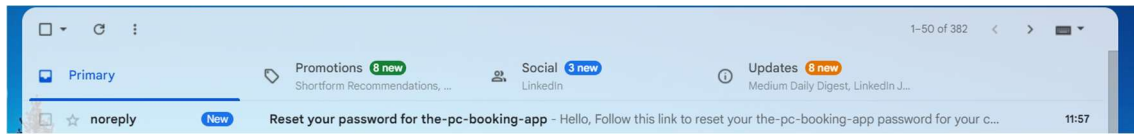


Figure 14. Reset Password email Inbox

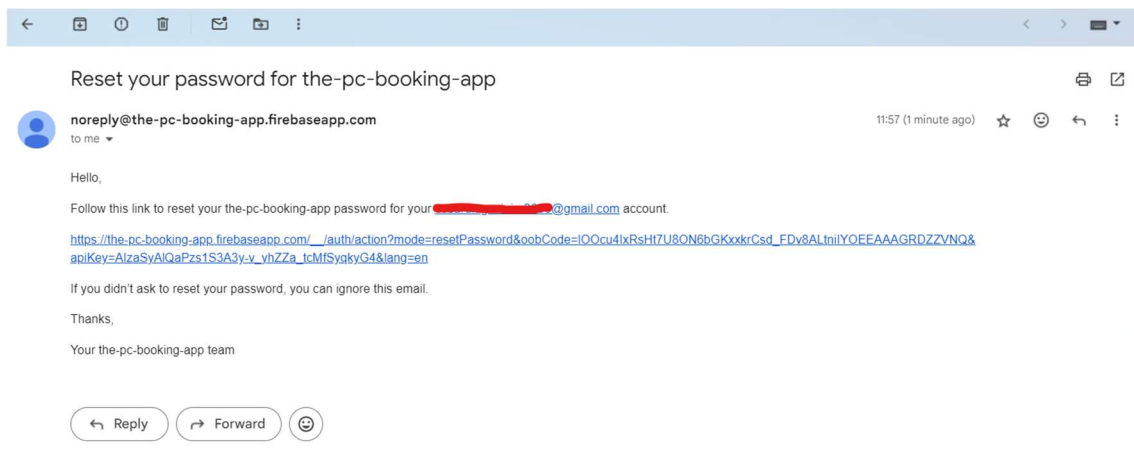


Figure 15. Reset Password Email Body

A screenshot of a password reset form. The title is 'Reset your password'. Below it, it says 'for [redacted]@gmail.com'. There is a text input field labeled 'New password' with a toggle icon (an eye) to its right. At the bottom right, there is a blue button labeled 'SAVE'.

Figure 16. Reset Your Password

Rationale:

Forgot Password is a feature of LoginActivity that allows users an easy way to regain access to their account. It presents an AlertDialog, which focuses on keeping the process isolated by entering a registered email, therefore not running them around in circles trying to have their password reset. Email validation helps to ensure that users enter their email addresses in the correct format, without errors, and it is properly processed for a reset request. Using Firebase Authentication for sending password reset emails, the system ensures the clients are directed through prompt and accurate instructions on their password reset procedure. Toast messages are used to provide an immediate answer on the reset of the state or request, so the users can know whether the email was actually sent successfully or if there was an error. With such a direct way of conveying, one ensures that user experience is good and it steers the user in the right way through the process of recovery.

Register Page (Register Activity)

If the user is not already registered, he can click the register button on the Log-in page to proceed to the RegisterActivity page where he can complete the registration process. Here, the application gathers user input, carries out the required validation checks, and uses Firebase Authentication to create a new user account. After a successful registration, the user's complete name is added to their profile and their details are saved in the Firebase Realtime Database.

```
private fun handleConfirmButtonClick() {
    val email = findViewById<EditText>(R.id.editTxtEmail).text.toString()
    val fullName = findViewById<EditText>(R.id.editTxtFullname).text.toString()
    val phoneNo = findViewById<EditText>(R.id.editTxtPhoneNumber).text.toString()
    val password = findViewById<EditText>(R.id.editTxtPassword).text.toString()
    val confirmPassword = findViewById<EditText>(R.id.editTxtConfirmPassword).text.toString()

    if (email.isBlank() || fullName.isBlank() || phoneNo.isBlank() || password.isBlank() || confirmPassword.isBlank()) {
        showToast("Please fill all fields")
        return
    }

    if (!android.util.Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
        showToast("Invalid email address")
        return
    }

    if (!phoneNo.matches("^\\d{6,15}$".toRegex())) {
        showToast("Invalid phone number")
        return
    }

    if (password != confirmPassword) {
        showToast("Passwords do not match")
        return
    }
}
```

Figure 17. Handle Registering

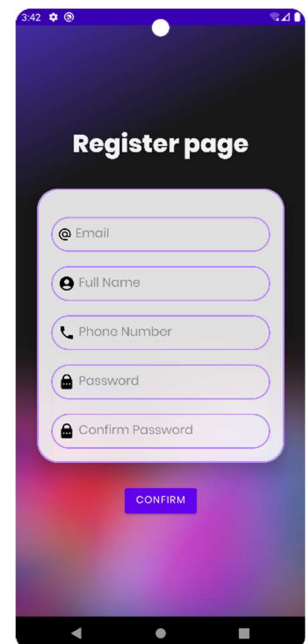


Figure 18. Register Page

Firstly, users provide their email, full name, phone number, password, and password confirmation through designated input fields. Then the application does the following validation checks:

- Ensures all fields are completed.
- Verifies the email format using `Patterns.EMAIL_ADDRESS`.
- Checks the phone number length to ensure it consists of 6 to 15 digits.
- Confirms that the password and confirmation password match.

If the user does not successfully pass all the validation checks, a toast message will be displayed as a way of providing the user with feedback on what mistake he committed.

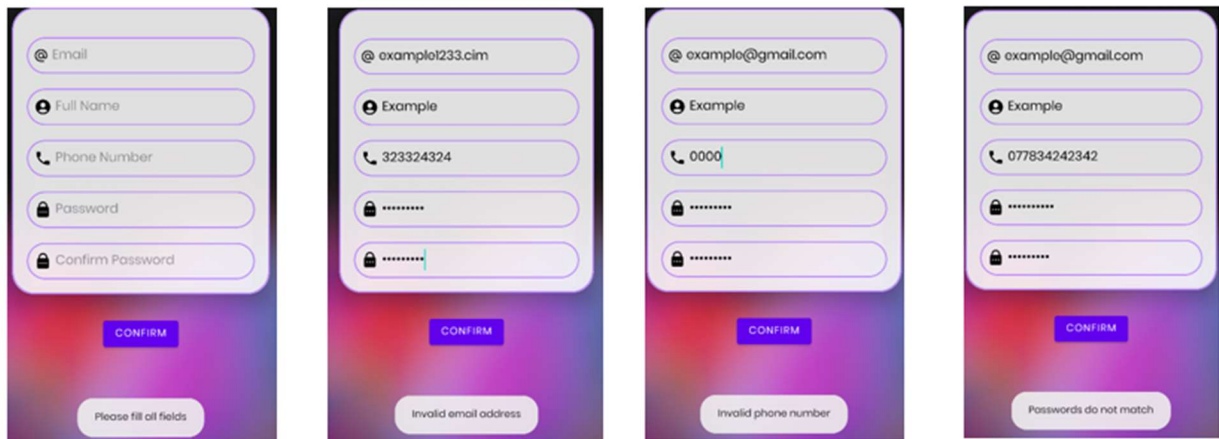


Figure 19. Register Error Toast Messages

If the validation is successful, the Firebase Authentication will create a new user account using the provided email and password.

```
auth.createUserWithEmailAndPassword(email, password)
    .addOnCompleteListener(this) { task ->
        if (task.isSuccessful) {
            val user = auth.currentUser
            val profileUpdates = UserProfileChangeRequest.Builder()
                .setDisplayName(fullName)
                .build()

            user?.updateProfile(profileUpdates)
                ?.addOnCompleteListener { profileUpdateTask ->
                    if (profileUpdateTask.isSuccessful) {
                        val database = FirebaseDatabase.getInstance().reference
                        val userData = User(email, fullName, phoneNo, admin: false) // Normal user
                        user?.uid?.let { userId ->
                            database.child(pathString: "users").child(userId).setValue(userData)
                                .addOnSuccessListener { it: Void?
                                    showToast("Registration successful")
                                    finish()
                                }
                                .addOnFailureListener { exception ->
                                    showToast("Failed to save user data: ${exception.message}")
                                }
                            }
                        } else {
                            showToast("Failed to update profile")
                        }
                    } else {
                        showToast("Registration failed: ${task.exception?.message}")
                    }
                }
    }
```

Figure 20. Register Firebase auth.

After the creation of the user account, there is the database storage of the user details, including email, full name, phone number, and role, in Firebase Realtime Database. The role by default is a normal user (admin: false).

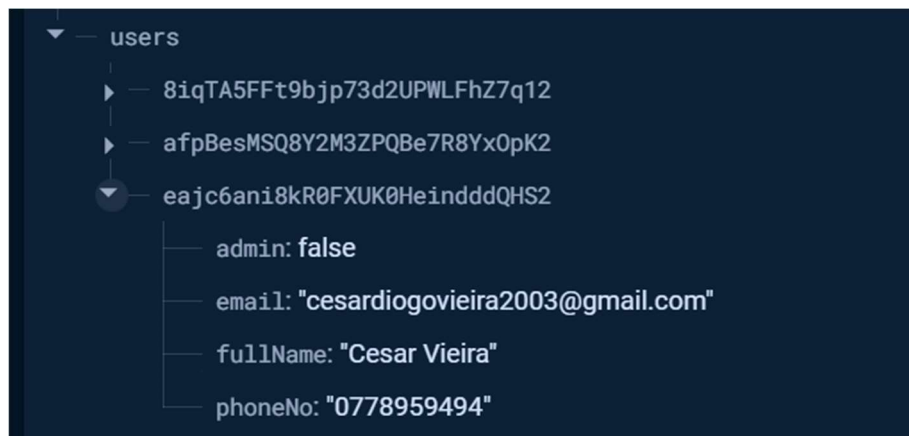


Figure 21. RealTime Database User Data Storage

Rationale:

The application's RegisterActivity is intended to make the registration process more secure and user-friendly. It validates all required fields (email, full name, phone number, and password), preventing incomplete submissions or incorrect data formats. The app uses Firebase Authentication to securely handle user account creation, password management, and profile updates, while Firebase Realtime Database is used to store additional user data.

The design includes validation checks for proper email format and password confirmation, which ensures data integrity and a more pleasant user experience. Feedback via Toast messages provides users with immediate information on the success or failure of their registration attempt, assisting them in resolving any issues quickly. This approach improves the user experience by maintaining clarity, security, and efficiency throughout the registration process.

User Process for Making a Booking

Overview

The booking process in the application is done to simplify and make it intuitive for the users to reserve a PC. The following are the steps starting from navigating into the booking section, including selecting a PC and time slot, and up to confirmation. These features include a 7-day window for booking, real-time refreshing, and feedback mechanisms for users' recommendations. This detailed explanation covers all stages of the booking process, what's involved at what stage, and the reasons behind that design.

Detailed Booking Process

1. Navigating to the Booking Section

Description: The user initiates the booking process by selecting the "Book" option from the app's navigation menu. This action directs them to the BookFragment, which displays a list of available PCs.

Implementation: A RecyclerView is used by the BookFragment to display a list of PCs. RecyclerView and PC data are connected via an adapter, which allows for seamless scrolling and interaction.

```
class BookFragment : Fragment() {  
  
    private var _binding: FragmentBookBinding? = null  
    @ Cesar Vieira  
    private val binding get() = _binding!!  
    private lateinit var bookViewModel: BookViewModel  
    private lateinit var pcAdapter: PcAdapter // Adapter for RecyclerView  
    @ Cesar Vieira  
    override fun onCreateView(  
        inflater: LayoutInflater,  
        container: ViewGroup?,  
        savedInstanceState: Bundle?  
    ): View {  
        _binding = FragmentBookBinding.inflate(inflater, container, attachToParent: false)  
        return binding.root  
    }  
    @ Cesar Vieira *  
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {  
        super.onViewCreated(view, savedInstanceState)  
        bookViewModel = ViewModelProvider(requireActivity()).get(BookViewModel::class.java)  
        // Initialize PcAdapter for RecyclerView  
        pcAdapter = PcAdapter(requireContext(), listOf()) { selectedPc ->  
            bookViewModel.setSelectedPc(selectedPc)  
            findNavController().navigate(R.id.action_bookFragment_to_availableDaysFragment)  
        }  
        // Set adapter and layout manager to RecyclerView  
        binding.pcsListView.layoutManager = LinearLayoutManager(requireContext())  
        binding.pcsListView.adapter = pcAdapter  
    }  
}
```

Figure 22. BookFragment

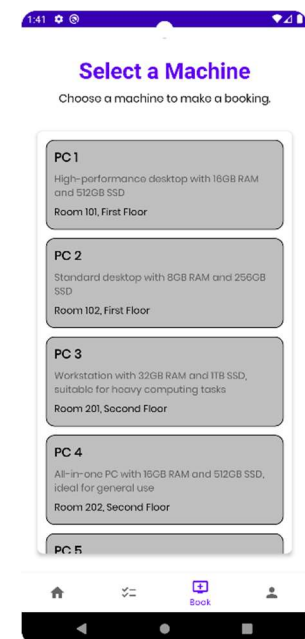


Figure 23. Book Page(Select a machine)

2. Selecting a PC

- Description: When the user decided on one of the PCs from the list, he will be navigated to AvailableDaysFragment. This fragment displays the days on which one of the chosen PCs is available for the upcoming 7 days.
- Step-by-step process: The AvailableDaysFragment fills the available days data in a RecyclerView via an adapter. It fetches this data by calling APIs based on which PC is selected.

```
override fun onCreateView(view: View, savedInstanceState: Bundle?) {
    super.onCreateView(view, savedInstanceState)

    bookViewModel = ViewModelProvider(requireActivity()).get(BookViewModel::class.java)

    // Initialize RecyclerView and Adapter for available days
    daysAdapter = AvailableDaysAdapter(requireContext()) { day ->
        // Handle day selection
        bookViewModel.setSelectedDay(day)
        findNavController().navigate(R.id.action_navigation_available_days_to_navigation_timeslot)
    }

    binding.daysRecyclerView.apply { this RecyclerView
        layoutManager = LinearLayoutManager(requireContext())
        adapter = daysAdapter
    }

    // Observe available days from ViewModel
    bookViewModel.availableDays.observe(viewLifecycleOwner) { availableDays ->
        // Update the list of available days in the adapter
        daysAdapter.submitList(availableDays)
    }

    // Fetch available days when fragment is created
    bookViewModel.fetchAvailableDays()
}
```

Figure 24. AvailableDaysFragment onCreateView function

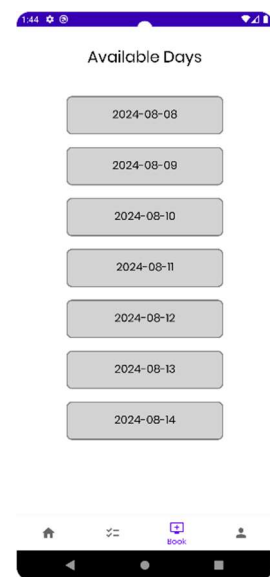


Figure 25. Available Days Page

3. Choosing an Available Day

- Description: After a user selects a particular day, they are navigated to the TimeSlotFragment; it allows a user to choose a specific slot in time on that day.

- Implementation: The TimeSlotFragment also uses a RecyclerView with an adapter to list the available time slots. This has the list updated automatically on input from the user.

```

override fun onCreateView(view: View, savedInstanceState: Bundle?) {
    super.onCreateView(view, savedInstanceState)
    bookViewModel = ViewModelProvider(requireActivity()).get(BookViewModel::class.java)
    // Initialize RecyclerView and Adapter for time slots
    timeSlotAdapter = TimeSlotAdapter(requireContext(), emptyList()) { timeSlot ->
        // Handle time slot selection and booking
        val pcId = bookViewModel.selectedPc.value?.id ?: ""
        val selectedDay = bookViewModel.selectedDay.value ?: ""
        val startTime = timeSlot.substringBefore( delimiter: "-").trim()
        val endTime = timeSlot.substringAfter( delimiter: "-").trim()
        // Convert time to minutes for database storage
        val startTimeInMinutes = startTime.split( _delimiters: ":")[0].toInt() * 60
        val endTimeInMinutes = endTime.split( _delimiters: ":")[0].toInt() * 60
        // Show confirmation dialog
        showConfirmationDialog(pcId, selectedDay, startTimeInMinutes, endTimeInMinutes)
    }
    binding.rvTimeSlots.apply { this: RecyclerView
        layoutManager = LinearLayoutManager(requireContext())
        adapter = timeSlotAdapter
    }
    // Observe available time slots from ViewModel
    bookViewModel.availableTimeSlots.observe(
        viewLifecycleOwner,
        Observer { availableTimeSlots ->
            timeSlotAdapter.submitList(availableTimeSlots)
        })
    // Fetch available time slots for the selected day and PC
    bookViewModel.selectedDay.observe(viewLifecycleOwner, Observer { selectedDay ->
        val selectedPc = bookViewModel.selectedPc.value

```

Figure 26. TimeSlotFragment onCreateView Function

4. Time Slot Selection

- Description: A user would select a time slot and a confirmation dialog would show up. The latter would ask the user to confirm the booking, making sure he/she gets the chance to review what has been selected.
- Implementation: The confirmation dialog is created using an AlertDialog. On confirmation of the booking, all the details are processed and saved onto the Firebase Realtime Database.

```

private fun showConfirmationDialog(
    pcId: String,
    selectedDay: String,
    startTimeInMinutes: Int,
    endTimeInMinutes: Int
) {
    val builder = AlertDialog.Builder(requireContext())
    builder.setTitle("Confirm Booking")
    builder.setMessage("Do you want to book this time slot?")

    builder.setPositiveButton( text: "Yes" ) { dialog, _ ->
        // Perform booking
        bookViewModel.bookPc(
            pcId,
            userId: FirebaseAuth.currentUser?.uid ?: "",
            selectedDay,
            startTimeInMinutes,
            endTimeInMinutes
        )
        dialog.dismiss()
    }

    builder.setNegativeButton( text: "No" ) { dialog, _ ->
        dialog.dismiss()
    }

    val dialog = builder.create()
    dialog.show()
}

```

Figure 28. TimeSlotFragment Confirmation dialog function

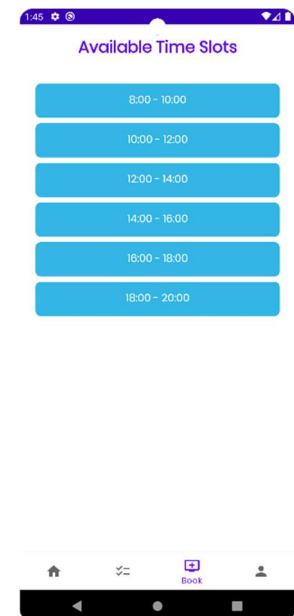


Figure 27. Available Time Slots Page

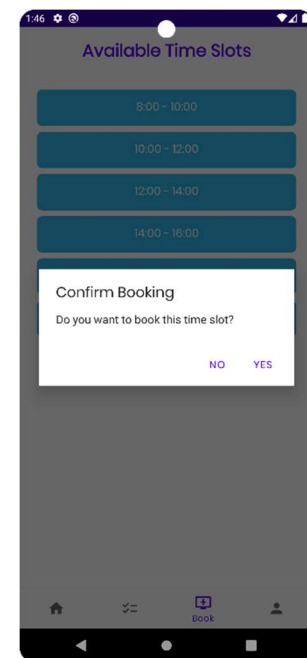


Figure 29. Time Slot Confirmation Dialog

5. Booking Confirmation

- Describe: After confirmation, booking details are passed into Firebase. Real-time feedback is given to the users by Toast messages of their booking status i.e. success in booking or failing.
- Implementation: The app utilizes Firebase Realtime Database to manage booking operations. The user is notified of actions taken, success and failure, by the use of Toast notifications.

```
fun bookPc(pcid: String, userId: String, date: String, startTime: Int, endTime: Int) {  
    val bookingId = bookingsRef.push().key ?: return  
    val booking = Booking(pcid, userId, date, startTime, endTime)  
  
    bookingsRef.child(bookingId).setValue(booking)  
    .addOnSuccessListener { // Void  
        // Fetch time slots to update list  
        fetchAvailableTimeSlots(selectedDay.value.toString(), selectedPc.value!!.id)  
        Toast.makeText(getApplication(), text "Booking successful.", Toast.LENGTH_SHORT).show()  
    }  
    .addOnFailureListener { // Exception  
        // Handle failure if needed  
        Toast.makeText(  
            getApplication(),  
            text "Something went wrong on your booking.",  
            Toast.LENGTH_SHORT  
        ).show()  
    }  
}
```

Figure 31. BookViewModel bookPc function

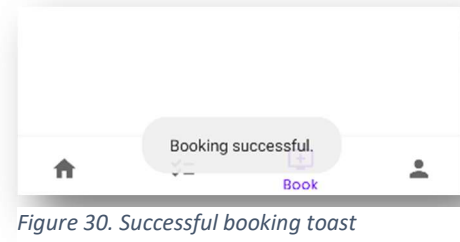


Figure 30. Successful booking toast

Behind the Scenes

Real-Time Booking Management

- Description: The Firebase Realtime Database will be used for managing booking data. Thus, all users will see the latest information about the availability of the PC, preventing double bookings and display of stale data.
- Implementation: The app listens to the real-time update for data on bookings and slots and reflects their state in the UI.

7-Day Booking Window

- Description: This app allows booking to be made within a 7-day window to plan resources and avoid any potential clashes in the future. This restriction is enforced by querying available days and time slots within a 7-day period.

- Implementation: Dynamic calculation of days available by the app; it filters out days beyond the 7-day window.



```

Cesar Vieira
fun fetchAvailableDays() {
    val daysInRange = mutableListOf<String>()
    val calendar = Calendar.getInstance()

    for (i in 0 until 7) {
        val date = calendar.time
        val dateFormat = SimpleDateFormat( pattern: "yyyy-MM-dd", Locale.getDefault())
        daysInRange.add(dateFormat.format(date))
        calendar.add(Calendar.DAY_OF_YEAR, amount: 1)
    }

    _availableDays.value = daysInRange
}

```

Figure 32. BookViewModel fetchAvailableDays function

Rationale:

The design of the booking process is oriented toward the following objectives:

User Experience:

- User-friendly Process: The process steps involved in booking are streamlined, with users following a simple procedure to pick a PC, available day, time slot, and therefore confirm their bookings. Such sequential methods eliminate guesswork fully, and hence they assist users fully in making their bookings.
- Real-Time Feedback: Providing immediate feedback using Toast messages and real-time updates assures the user of their status of booking, thereby enhancing the overall user experience. It makes sure that users can resolve any problem or even confirm a booking with minimal delay.
- 7-Day Window: Bookings would be constrained within a 7-day window. This would keep availability in check and secure no overbooking instance occurs. This short-term restriction will balance user flexibility with appropriate constraints while respecting effective management of resources, to ensure a resource holds are maintained only within a manageable timeframe.

Technical Efficiency:

- Firebase Integration: Retrieving data on bookings via the Firebase Realtime Database guarantees consistency in data for all devices, which is important to prevent double bookings and for availability. At the same time, real-time synchronization minimizes the occurrence of double-bookings and outdated availability, making a booking system more reliable.
- Adapters and RecyclerViews: Adapters and RecyclerViews ensure very efficient data handling to enable perfect scrolling in the display of PCs, days available, and time slots. Views are bound to adapters in such a way that dynamic data updating and user interactions are present, hence providing smooth user experience in general. Such a well-structured booking process entails providing the end users with a seamless, effective manner of reserving PCs, hence maintaining reliability of the system and user satisfaction.


```

class AvailableDaysAdapter(
    private val context: Context,
    private val onItemClick: (String) -> Unit
) : RecyclerView.Adapter<AvailableDaysAdapter.AvailableDayViewHolder>() {

    private var availableDays: List<String> = emptyList()

    @ Cesar Vieira
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): AvailableDayViewHolder {
        val view = LayoutInflater.from(context).inflate(R.layout.item_available_day, parent, attachToRoot: false)
        return AvailableDayViewHolder(view)
    }

    @ Cesar Vieira
    override fun onBindViewHolder(holder: AvailableDayViewHolder, position: Int) {
        val day = availableDays[position]
        holder.bind(day)
        holder.itemView.setOnClickListener { @View()
            onItemClick(day)
        }
    }

    @ Cesar Vieira
    override fun getItemCount(): Int {
        return availableDays.size
    }

    @ Cesar Vieira
    fun submitList(days: List<String>) {
        availableDays = days
    }
}

```

Figure 33. AvailableDaysAdapter

```

class PcAdapter(
    private val context: Context,
    private var pcsList: List<Pc>,
    private val onItemClick: (Pc) -> Unit
) : RecyclerView.Adapter<PcAdapter.PcViewHolder>() {

    @ Cesar Vieira
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): PcViewHolder {
        val view = LayoutInflater.from(context).inflate(R.layout.item_pc, parent, attachToRoot: false)
        return PcViewHolder(view)
    }

    @ Cesar Vieira
    override fun onBindViewHolder(holder: PcViewHolder, position: Int) {
        val pc = pcsList[position]
        holder.bind(pc)
        holder.itemView.setOnClickListener { @View()
            onItemClick(pc)
        }
    }

    @ Cesar Vieira
    override fun getItemCount(): Int {
        return pcsList.size
    }
}

```

Figure 34.PcAdapter

Manage Bookings Process

Overview

The Manage Bookings process helps the user manage his bookings for PCs effectively. The user can view, modify, or cancel bookings. This feature is divided into two major categories: Future Bookings and Past Bookings. This can be viewed in all the upcoming bookings besides canceling them in case someone wishes to, likewise gaining access to history of prior bookings. This project does use the Firebase Realtime Database, hence ensuring that at all times, the information on booking is updated and synchronized at any time to any device. Very important in this choice of database is the maintenance of the integrity or reliability of the booking system, thereby making it friendly to its users.

Detailed Booking Management Process

1. Navigating to Manage Bookings:

Description: Users start the booking management procedure by selecting the ManageFragment option from the app's main navigation menu. This fragment will then provide a list of all upcoming bookings as well as a button to see past bookings.

Implementation: The ManageFragment uses a RecyclerView to display current bookings, which are bound and displayed using a BookingAdapter. To view previous reservations, users can travel to the PastBookingsActivity by clicking a button.

```
class ManageFragment : Fragment() {
    private var _binding: FragmentManageBinding? = null
    // Cesar Vieira
    private val binding get() = _binding!!
    private lateinit var manageViewModel: ManageViewModel
    private lateinit var bookingAdapter: BookingAdapter

    // Cesar Vieira
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentManageBinding.inflate(inflater, container, attachToParent: false)
        return binding.root
    }

    // Cesar Vieira
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        manageViewModel = ViewModelProvider( owner: this).get(ManageViewModel::class.java)
        bookingAdapter = BookingAdapter(emptyList()) { booking ->
            manageViewModel.unbook(booking)
        }
        binding.rvBookings.apply { this: RecyclerView
            layoutManager = LinearLayoutManager(requireContext())
            adapter = bookingAdapter
        }
        manageViewModel.futureBookings.observe(viewLifecycleOwner) { bookings ->
            bookingAdapter.updateBookings(bookings)
        }
    }
}
```

Figure 35. ManageFragment

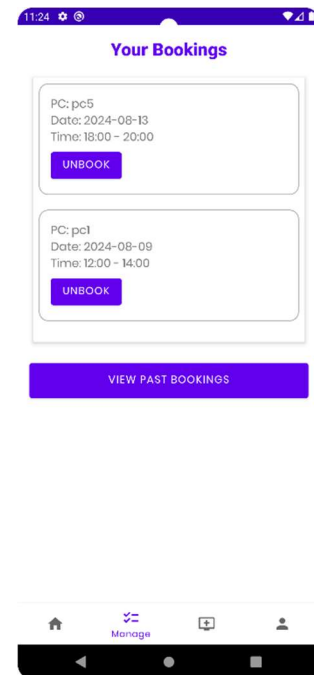


Figure 36. Manage Bookings Page

2. Viewing Current Bookings

Description: Users can see a list of their upcoming bookings, including details such as PC ID, date, and time range. Each booking item includes an option to cancel the booking if it is still in the future.

Implementation: The BookingAdapter within the RecyclerView binds booking details to the view. It also manages user interactions like cancelling bookings.

```
class BookingViewHolder(
    itemView: View,
    private val onUnbookClick: (Booking) -> Unit,
    private val context: Context
) : RecyclerView.ViewHolder(itemView) {
    private val tvBookingDetails: TextView = itemView.findViewById(R.id.tvBookingDetails)
    private val btnUnbook: Button = itemView.findViewById(R.id.btnUnbook)

    // Cesar Vieira
    fun bind(booking: Booking) {
        val details =
            "PC: ${booking.pcId}\nDate: ${booking.date}\nTime: ${formatTime(booking.startTime)} - ${formatTime(booking.endTime)}"
        tvBookingDetails.text = details
        btnUnbook.visibility = if (isFutureBooking(booking)) View.VISIBLE else View.GONE
        btnUnbook.setOnClickListener { @View()
            showConfirmationDialog(booking)
        }
    }
}
```

Figure 38. Booking Adapter ViewHolder

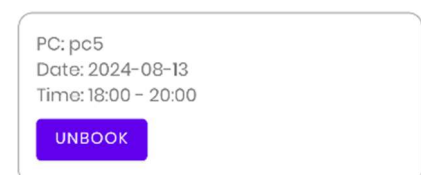


Figure 37. Manage Bookings RecyclerView Item

3. Cancelling a Booking

Description: When a user chooses to cancel a booking, a confirmation dialog appears to verify the cancellation. Upon confirmation, the booking is removed from the list and updated in the Firebase Realtime Database.

Implementation: The BookingAdapter displays a confirmation dialog using AlertDialog. If confirmed, the ManageViewModel processes the cancellation and updates the Firebase database.

```
private fun showConfirmationDialog(booking: Booking) {
    AlertDialog.Builder(context) AlertDialog.Builder
        .setTitle("Confirm Unbooking") AlertDialog.Builder
        .setMessage("Are you sure you want to unbook this time slot?")
        .setPositiveButton(text: "Yes") { _, _ ->
            onUnbookClick(booking)
        }
        .setNegativeButton(text: "No", listener: null)
        .show()
}
```

Figure 40. Booking adapter showConfirmationDialog function

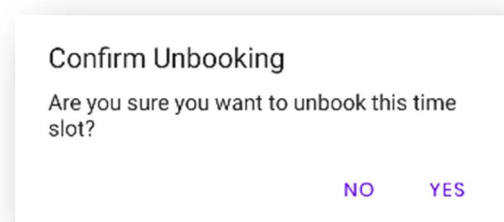


Figure 39. Confirmation dialog for unbooking a reservation

4. Viewing Past Bookings

Description: To view past bookings, users click the "Past Bookings" button in the ManageFragment, which navigates them to the PastBookingsActivity. This activity displays a list of past bookings.

Implementation: The PastBookingsActivity uses a RecyclerView with a BookingAdapter to display past bookings. It fetches the data from the ManageViewModel.

```
class PastBookingsActivity : AppCompatActivity() {

    private lateinit var binding: ActivityPastBookingsBinding
    private val manageViewModel: ManageViewModel by viewModels()
    private lateinit var pastBookingAdapter: BookingAdapter

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityPastBookingsBinding.inflate(layoutInflater)
        setContentView(binding.root)

        pastBookingAdapter = BookingAdapter(emptyList()) {}

        binding.rvPastBookings.apply { this: RecyclerView
            layoutManager = LinearLayoutManager(context: this@PastBookingsActivity)
            adapter = pastBookingAdapter
        }

        manageViewModel.pastBookings.observe(owner: this, Observer { bookings ->
            pastBookingAdapter.updateBookings(bookings)
        })

        manageViewModel.fetchUserBookings()
    }
}
```

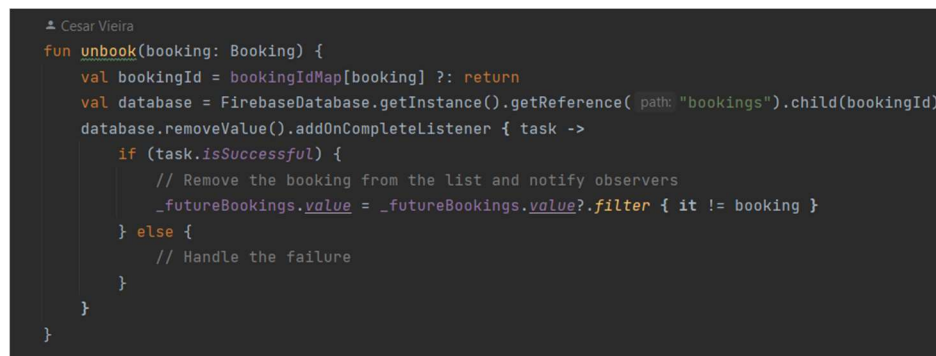


Figure 41. Past Bookings Page

5. Updating Booking Data

Description: After cancelling a booking, the data is updated in Firebase and the UI is refreshed to reflect the latest information. The list of future bookings is updated accordingly.

Implementation: The ManageViewModel communicates with Firebase to remove the booking and update the list of future bookings. Real-time updates ensure the UI remains accurate.

A screenshot of a code editor showing the implementation of the `unbook` function in the `ManageViewModel` class. The code is written in Kotlin and is attributed to Cesar Vieira. It shows how a booking is removed from the database and the UI list is updated using a filter.

```
fun unbook(booking: Booking) {
    val bookingId = bookingIdMap[booking] ?: return
    val database = FirebaseDatabase.getInstance().getReference(path: "bookings").child(bookingId)
    database.removeValue().addOnCompleteListener { task ->
        if (task.isSuccessful) {
            // Remove the booking from the list and notify observers
            _futureBookings.value = _futureBookings.value?.filter { it != booking }
        } else {
            // Handle the failure
        }
    }
}
```

Figure 43. ManageViewModel unbook function

Behind the Scenes

Real-Time Data Synchronization:

Description: Real-time management of booking data is done in the application with the Firebase Realtime Database, ensuring that no cases of double bookings or users experiencing issues with viewing outdated information occur.

Implementation: Firebase is listening for real-time updates, and it synchronizes data regarding bookings across all devices, updating the UI dynamically to show changes.

Efficient Data Handling:

Description: The use of RecyclerView and BookingAdapter ensures that the app handles large lists of bookings efficiently. Adapters bind data to UI elements in a way that supports smooth scrolling and dynamic updates.

Implementation: Adapters manage the display of booking items and user interactions, such as unbooking, by efficiently updating the RecyclerView as data changes.

Rationale

User Experience:

Guided Interaction: The process is designed to guide users through managing their bookings efficiently. By displaying current bookings and providing an easy way to cancel future reservations, users can quickly and effectively manage their reservations.

Real-Time Feedback: With real-time updates and confirmatory dialogs, the user cannot but be aware of exactly what he is doing, getting immediate feedback for every action.

Technical Efficiency:

Firestore Integration: The Firestore Real-time Database is used to ensure consistency in the data of the booking, which enables real-time updating across all devices. Thus, the problems of double bookings and display of outdated information are avoided.

Adapters and RecyclerViews: These are the components that deal with the datasets efficiently, allowing smooth and responsive user interactions over booking data.

5.2 Admin Major components

We have covered how the product will meet customer requirements in terms of user-facing features and functionality. Now let's pivot to aspects of the application that have to do with administration. The rest of this paper takes a deeper look into major components aimed at administrative management.

Admin Manage Bookings: This aspect enables administrators to monitor and oversee booking schedules made by users hence facilitating efficient reservation and resource management.

Admin Manage Users: This section allows an administrator to oversee profiles, roles, permissions, ensuring user data is up-to-date and access levels are accurate.

Admin Manage Machines: It concentrates on controlling system machines including updating machine information.

Every part mentioned before plays a vital role for maintaining integrity and smooth operation of the app. Each section below will highlight the functions, technical aspects as well as the way people use these elements as per their different needs in details in relation to every component.

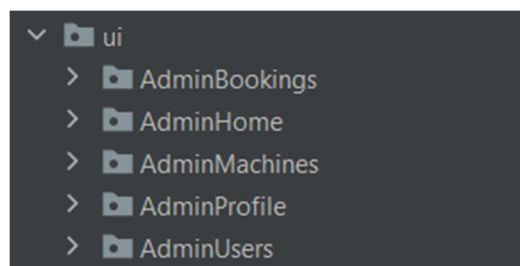


Figure 44. Admin Packages

Admin Manage Bookings

Overview

The purpose of the Admin Manage Bookings component is to give administrators effective tools for monitoring and controlling user bookings inside the application. This section describes this component's technical implementations and functionalities, including how it views, filters, and manages bookings and the thinking behind the design decisions.

Detailed Booking Management Process

1. Viewing Bookings

- **Description:** The AdminBookingsFragment allows administrators to access an exhaustive list of all bookings. This fragment gives a clear overview of booking details, like PC ID, user name, date, and time slots, by displaying bookings in a RecyclerView.

- **Implementation:** A BookingAdapter is used by the AdminBookingsFragment to display booking data that is retrieved via the AdminBookingsViewModel. By connecting the booking data to the RecyclerView, this adapter ensures effective data handling and smooth scrolling.

```
class AdminBookingsFragment : Fragment() {
    private var _binding: FragmentAdminBookingsBinding? = null
    @ Cesar Vieira
    private val binding get() = _binding!!
    private val viewModel: AdminBookingsViewModel by viewModels()
    private val dateFormat = SimpleDateFormat(pattern = "yyyy-MM-dd", Locale.getDefault())
    @ Cesar Vieira
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentAdminBookingsBinding.inflate(inflater, container, attachToParent: false)
        return binding.root
    }
    @ Cesar Vieira
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        val adapter = BookingAdapter(emptyList()) { bookingPair ->
            val (bookingId, booking) = bookingPair
            val intent = Intent(requireContext(), BookingDetailsActivity::class.java).apply { this.intent
                putExtra(name = "BOOKING_ID", bookingId)
                putExtra(name = "PC_ID", booking.pcId)
                putExtra(name = "USER_ID", booking.userId) // Pass USER_ID
                putExtra(name = "DATE", booking.date)
                putExtra(name = "START_TIME", booking.startTime)
                putExtra(name = "END_TIME", booking.endTime)
            }
        }
    }
}
```

Figure 45. AdminBookingsFragment

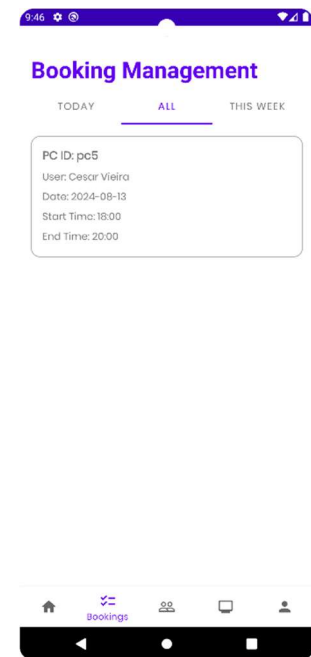


Figure 46. Admin Booking Management page

2. Filtering Bookings

- **Description:** Using a TabLayout, administrators can use the AdminBookingsFragment to filter bookings according to several timeframes, including today, this week, and all bookings.
- **Implementation:** The fragment includes a TabLayout with tabs for each timeframe. The selected tab determines the filter applied to the booking list. The updateBookings method filters bookings based on the selected tab and updates the BookingAdapter accordingly.

```
binding.tabLayout.addOnTabSelectedListener(object : TabLayout.OnTabSelectedListener {
    override fun onTabSelected(tab: TabLayout.Tab?) {
        tab?.let { it: TabLayout.Tab
            updateBookings(adapter, bookings: viewModel.bookings.value ?: emptyList())
        }
    }
})

override fun onTabUnselected(tab: TabLayout.Tab?) {}
override fun onTabReselected(tab: TabLayout.Tab?) {}
})

@ Cesar Vieira
private fun updateBookings(adapter: BookingAdapter, bookings: List<Pair<String, Booking>>) {
    val todayDate = getTodayDate()
    val nextWeekDate = getNextWeekDate()
    val filteredBookings = when (binding.tabLayout.selectedTabPosition) {
        0 -> bookings.filter { it.second.date == todayDate }
        1 -> bookings.filter { it.second.date >= todayDate && it.second.date <= nextWeekDate }
        2 -> bookings.filter { isThisWeek(it.second.date) }
        else -> bookings
    }
    adapter.updateBookings(filteredBookings)
}
```

Figure 48. AdminBookingsFragment updateBookings function

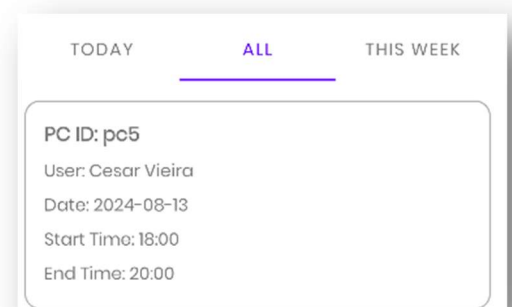


Figure 47. Admin Manage bookings TabLayout

3. Viewing Booking Details

- **Description:** The BookingDetailsActivity allows administrators to view detailed information about a booking. This activity displays detailed information about the booking, including user information and the option to cancel it.
- **Implementation:** BookingDetailsActivity receives booking information via Intent extras. It retrieves additional user information from Firebase and displays all applicable booking information. A confirmation dialogue is used to deal with booking cancellations.

```
class BookingDetailsActivity : AppCompatActivity() {
    private lateinit var bookingId: String
    private lateinit var userId: String
    private lateinit var dbRef: DatabaseReference

    // Cesar Vieira *
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_booking_details)

        // Initialize Firebase Database Reference
        dbRef = FirebaseDatabase.getInstance().reference
        // Retrieve the booking details from the Intent
        bookingId = intent.getStringExtra( name: "BOOKING_ID") ?: ""
        val pcId = intent.getStringExtra( name: "PC_ID") ?: ""
        userId = intent.getStringExtra( name: "USER_ID") ?: ""
        val date = intent.getStringExtra( name: "DATE") ?: ""
        val startTime = intent.getIntExtra( name: "START_TIME", defaultValue: 0)
        val endTime = intent.getIntExtra( name: "END_TIME", defaultValue: 0)

        // Populate the UI with the booking details
        findViewById<TextView>(R.id.tvPcId).text = pcId
        findViewById<TextView>(R.id.tvDate).text = date
        findViewById<TextView>(R.id.tvStartTime).text = convertMinutesToHours(startTime)
        findViewById<TextView>(R.id.tvEndTime).text = convertMinutesToHours(endTime)
        findViewById<TextView>(R.id.tvBookingId).text = bookingId
        findViewById<TextView>(R.id.tvUserId).text = userId
        // Fetch user name from Firebase
        fetchUserName(userId) { userName ->
            findViewById<TextView>(R.id.tvUserFullName).text = userName
        }
    }
}
```

Figure 50. BookingDetailsActivity

The screenshot shows the Booking Details Page with the following fields and values:

- PC ID: pc5
- User ID: ecjcb8ani8kR0FXUK0HeindddQHS2
- User Name: Cesar Vieira
- Booking ID: -O3j_BFTWGVlI2qo2SA5
- Date: 2024-08-13
- Start Time: 18:00
- End Time: 20:00

A purple button labeled "CANCEL BOOKING" is located at the bottom right of the form.

Figure 51. Booking Details Page

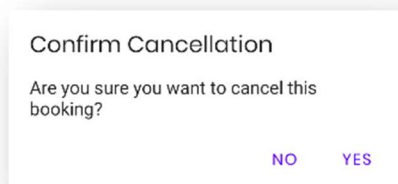


Figure 49. Confirm Booking Cancellation toast.

Behind the Scenes

Real-Time Booking Management

- **Description:** Firebase Realtime Database is used to manage the booking data, guaranteeing that all users see the most up-to-date information about PC availability. This stops out-of-date data and duplicate bookings.

- **Implementation:** The AdminBookingsViewModel dynamically updates the user interface (UI) in response to changes in booking data by listening for real-time updates. Administrators will always have access to the most recent booking data thanks to this.

```
class AdminBookingsViewModel : ViewModel() {

    private val _bookings = MutableLiveData<List<Pair<String, Booking>>>()
    val bookings: LiveData<List<Pair<String, Booking>>> = _bookings

    private val _users = MutableLiveData<Map<String, User>>()
    val users: LiveData<Map<String, User>> = _users

    init {
        fetchBookings()
        fetchUsers()
    }

    private fun fetchBookings() {
        val database = FirebaseDatabase.getInstance().getReference<String>("bookings")
        database.addValueEventListener(object : ValueEventListener {
            override fun onDataChange(snapshot: DataSnapshot) {
                val bookingsList = mutableListOf<Pair<String, Booking>>()
                val currentDate = SimpleDateFormat("yyyy-MM-dd", Locale.getDefault()).format(Date())

                for (bookingSnapshot in snapshot.children) {
                    val bookingId = bookingSnapshot.key ?: continue
                    val booking = bookingSnapshot.getValue(Booking::class.java) ?: continue

                    if (booking.date >= currentDate) {
                        bookingsList.add(Pair(bookingId, booking))
                    }
                }

                _bookings.value = bookingsList
            }
        })
    }
}
```

Figure 52. AdminBookingsViewModel

Rationale

The design and implementation of the Admin Manage Bookings component are driven by the need for efficient administrative control and a seamless user experience.

User Experience

- **Intuitive Filtering:** The use of TabLayout for filtering bookings by different timeframes (today, this week, next week) simplifies the process for administrators, making it easy to locate relevant bookings.
- **Detailed Insights:** Detailed booking information and user details are provided to ensure that administrators make informed decisions on how to manage bookings effectively.
- **Real-Time Feedback:** Integration with Firebase ensures that booking data is updated in real-time, allowing administrators to work with the most current information and avoid issues such as double bookings.

Technical Efficiency

- **Separation of Concerns:** The use of distinct classes for data handling (AdminBookingsViewModel), UI display (BookingAdapter), and user interaction

(AdminBookingsFragment and BookingDetailsActivity) makes for good architecture and maintainability.

- **LiveData and ViewModel:** Utilizing Android's LiveData and ViewModel architecture components facilitates responsive UI updates and lifecycle-aware data management, ensuring efficient data handling and user interactions.
- **Firebase Integration:** The use of Firebase Realtime Database for bookings will give consistency to the data stored and at the same time assuring scaling. Due to Firebase's connectivity in real-time, this makes it a great selection for this component. It gives assurance of reliable data management, avoiding double-booking.

Admin Manage Users

Overview

The Admin Manage Users component is designed to provide administrators with comprehensive tools for overseeing and managing user accounts within the application. This section outlines the functionalities and technical implementations of this component, detailing the processes for viewing, filtering, and managing users, as well as the design choices behind these functionalities.

Detailed User Management Process

1. Viewing Users

Description: Administrators can view a comprehensive list of all users through the AdminUsersFragment. This fragment displays users in a RecyclerView, providing a clear overview of user details such as user ID, name, and role.

Implementation: The AdminUsersFragment fetches user data via the AdminUsersViewModel and displays it using a UsersAdapter. This adapter binds the user data to the RecyclerView, ensuring efficient data handling and smooth scrolling.

```
class AdminUsersFragment : Fragment() {  
  
    private var _binding: FragmentAdminUsersBinding? = null  
    private val binding get() = _binding!!  
    private val viewModel: AdminUsersViewModel by viewModels()  
  
    override fun onCreateView(  
        inflater: LayoutInflater, container: ViewGroup?,  
        savedInstanceState: Bundle?  
    ): View {  
        _binding = FragmentAdminUsersBinding.inflate(inflater, container, attachToParent: false)  
        return binding.root  
    }  
  
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {  
        super.onViewCreated(view, savedInstanceState)  
  
        val adapter = UsersAdapter(emptyList()) { userPair ->  
            val (userId, user) = userPair  
            val intent = Intent(requireContext(), UserDetailsActivity::class.java).apply { this.intent.putExtra(name: "USER_ID", userId)  
                putExtra(name: "USER_NAME", user.fullName)  
                putExtra(name: "USER_EMAIL", user.email)  
                putExtra(name: "USER_PHONE", user.phoneNo)  
                putExtra(name: "USER_IS_ADMIN", user.admin)  
            }  
        }  
    }  
}
```

Figure 54. AdminUsersFragment

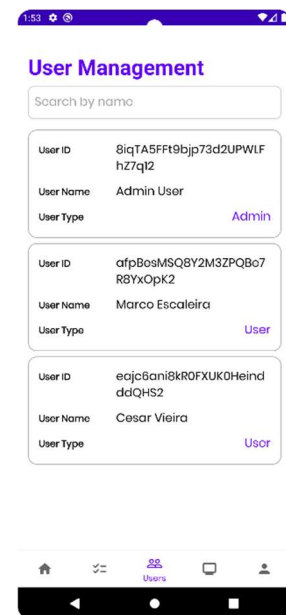


Figure 53. User Management Page

2. Filtering Users

- Description:** The AdminUsersFragment allows administrators to filter users by typing into a search bar. The list updates dynamically to show users whose names match the search query.

- **Implementation:** The fragment includes a searchBar that uses addTextChangedListener to listen for text input. The filter method of the UsersAdapter filters the user list based on the search query and updates the RecyclerView.

```
binding.searchBar.addTextChangedListener { text ->
    adapter.filter(text.toString())
}
}
```

Figure 56. Search filter implementation.

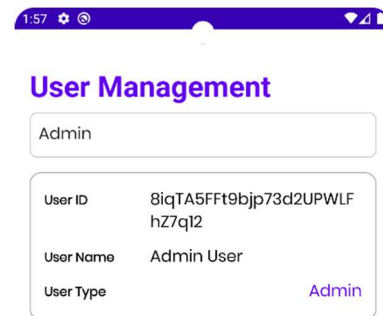


Figure 55. Searching example

3. Viewing User Details

- **Description:** Administrators can click on a user to view detailed information in the UserDetailsActivity. This activity displays comprehensive details about the user, including name, email, phone number, and role.
- **Implementation:** The UserDetailsActivity receives user details via Intent extras. It displays these details and provides options to update the user's role or delete the user.

```
class UserDetailsActivity : AppCompatActivity() {
    private lateinit var userId: String
    private lateinit var userName: String
    private lateinit var userEmail: String
    private lateinit var userPhone: String
    private var isAdmin: Boolean = false

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_user_details)

        userId = intent.getStringExtra( name: "USER_ID") ?: ""
        userName = intent.getStringExtra( name: "USER_NAME") ?: ""
        userEmail = intent.getStringExtra( name: "USER_EMAIL") ?: ""
        userPhone = intent.getStringExtra( name: "USER_PHONE") ?: ""
        isAdmin = intent.getBooleanExtra( name: "USER_IS_ADMIN", defaultValue: false)

        findViewById<TextView>(R.id.userIdTextView).text = "User ID: $userId"
        findViewById<TextView>(R.id.userNameTextView).text = "Name: $userName"
        findViewById<TextView>(R.id.userEmailTextView).text = "Email: $userEmail"
        findViewById<TextView>(R.id.userPhoneTextView).text = "Phone: $userPhone"
        findViewById<TextView>(R.id.userRoleTextView).text =
            if (isAdmin) "Role: Admin" else "Role: User"

        findViewById<Button>(R.id.updateRoleButton).setOnClickListener { updateUserRole() }
        findViewById<Button>(R.id.deleteUserButton).setOnClickListener { deleteUser() }
    }
}
```

Figure 58. User Details Activity

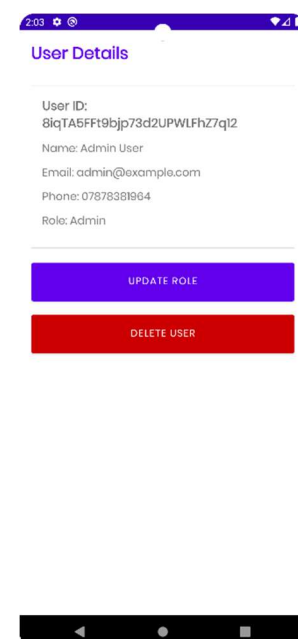


Figure 57. User Details Page

4. Updating User Role

- **Description:** Within the UserDetailsActivity, administrators can update the role of a user (e.g., from user to admin or vice versa) using the updateRoleButton.

- **Implementation:** The `updateRoleButton` triggers the `updateUserRole` method, which updates the user's role in the Firebase Realtime Database. A Toast message confirms the update's success or failure.

```
private fun updateUserRole() {
    val database = FirebaseDatabase.getInstance().getReference( path: "users/$userId")
    val updatedUser = User(userName, userEmail, userPhone, !isAdmin)
    database.setValue(updatedUser).addOnCompleteListener { it:Task<Void!>
        if (it.isSuccessful) {
            Toast.makeText( context: this, text: "User role updated", Toast.LENGTH_SHORT).show()
            finish()
        } else {
            Toast.makeText( context: this, text: "Failed to update user role", Toast.LENGTH_SHORT).show()
        }
    }
}
```

Figure 60. UserDetailsActivity updateUserRole function

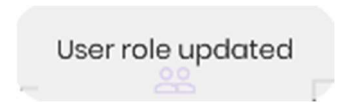


Figure 59. User Role update success toast

5. Deleting a User

- **Description:** Administrators can delete a user using the `deleteUserButton` in the `UserDetailsActivity`.
- **Implementation:** The `deleteUserButton` triggers the `deleteUser` method, which removes the user from the Firebase Realtime Database. A Toast message confirms the deletion's success or failure.
- **Visual:** Include an image of the `deleteUserButton` in the `UserDetailsActivity`.

```
private fun deleteUser() {
    val database = FirebaseDatabase.getInstance().getReference( path: "users/$userId")
    database.removeValue().addOnCompleteListener { it:Task<Void!>
        if (it.isSuccessful) {
            Toast.makeText( context: this, text: "User deleted", Toast.LENGTH_SHORT).show()
            finish()
        } else {
            Toast.makeText( context: this, text: "Failed to delete user", Toast.LENGTH_SHORT).show()
        }
    }
}
```

Figure 61. UserDetailsActivity deleteUser function

Behind the Scenes

Real-Time User Management

- **Description:** The user data is managed using Firebase Realtime Database, ensuring that all users see the most current information. This prevents outdated data and ensures consistency across all devices.

- **Implementation:** The AdminUsersViewModel listens for real-time updates on user data and dynamically updates the UI to reflect any changes. This ensures administrators always have access to the latest user information.

```
class AdminUsersViewModel : ViewModel() {
    private val _users = MutableLiveData<List<Pair<String, User>>>()
    val users: LiveData<List<Pair<String, User>>> = _users

    @Cesar Vieira
    init {
        fetchUsers()
    }

    @Cesar Vieira
    private fun fetchUsers() {
        val database = FirebaseDatabase.getInstance().getReference<Path>("users")
        database.addValueEventListener(object : ValueEventListener {
            override fun onDataChange(snapshot: DataSnapshot) {
                val userList = mutableListOf<Pair<String, User>>()
                for (userSnapshot in snapshot.children) {
                    val userId = userSnapshot.key ?: continue
                    val user = userSnapshot.getValue(User::class.java) ?: continue
                    userList.add(Pair(userId, user))
                }
                _users.value = userList
            }

            override fun onCancelled(error: DatabaseError) {
                // Handle database error
            }
        })
    }
}
```

Figure 62. AdminUsersViewModel

Rationale

The Admin Manage Users component shall be designed and developed around the principles of efficient administrative control and seamless user experience.

User Experience

- **Intuitive Filtering:** The search functionality in the AdminUsersFragment allows administrators to quickly find users by name, simplifying user management.
- **Detailed Insights:** Providing detailed user information and management options ensures administrators can make informed decisions and manage user accounts effectively.
- **Real-Time Feedback:** Integration with Firebase ensures that user data is updated in real-time, allowing administrators to work with the most current information and avoid issues such as outdated data.

Technical Efficiency

- **Separation of Concerns:** The use of distinct classes for data handling (AdminUsersViewModel), UI display (UsersAdapter), and user interaction (AdminUsersFragment and UserDetailsActivity) promotes a clean architecture and maintainability.

- **LiveData and ViewModel:** Utilizing Android's LiveData and ViewModel architecture components facilitates responsive UI updates and lifecycle-aware data management, ensuring efficient data handling and user interactions.
- **Firebase Integration:** Using Firebase Realtime Database for managing user data ensures data consistency and scalability. Firebase's real-time synchronization capabilities make it an ideal choice for this component, providing reliable data management and preventing outdated data.

Admin Manage Machines

Overview

The Admin Manage Machines component enables administrators to efficiently manage machine records within the application. This includes viewing, creating, updating, and deleting machine entries. This section provides a detailed breakdown of the functionalities and the technical implementation of this component, emphasizing user interface design and interaction logic.

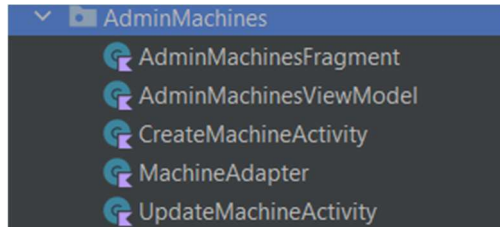


Figure 63. Admin machines package

Detailed Admin Machine Management Process

1. Viewing Machines

- **Description:** Administrators can view a list of all machines via the AdminMachinesFragment. This fragment displays machine details in a RecyclerView.
- **Implementation:** The AdminMachinesFragment fetches machine data through the AdminMachinesViewModel and displays it using a MachineAdapter. This adapter binds the machine data to the RecyclerView, ensuring efficient data handling.

```
class AdminMachinesFragment : Fragment() {  
  
    private var _binding: FragmentAdminMachinesBinding? = null  
    private val binding get() = _binding!!  
    private val viewModel: AdminMachinesViewModel by viewModels()  
  
    override fun onCreateView(  
        inflater: LayoutInflater, container: ViewGroup?,  
        savedInstanceState: Bundle?  
    ): View {  
        _binding = FragmentAdminMachinesBinding.inflate(inflater, container, attachToParent: false)  
        return binding.root  
    }  
  
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {  
        super.onViewCreated(view, savedInstanceState)  
  
        // Initialize the adapter with an empty list and set the click listener  
        val adapter = MachineAdapter(emptyList()) { machine ->  
            val intent = Intent(requireContext(), UpdateMachineActivity::class.java).apply {  
                putExtra(name: "machineId", machine.id)  
                putExtra(name: "machineName", machine.name)  
                putExtra(name: "machineDescription", machine.description)  
            }  
        }  
    }  
}
```

Figure 64. AdminMachinesFragment

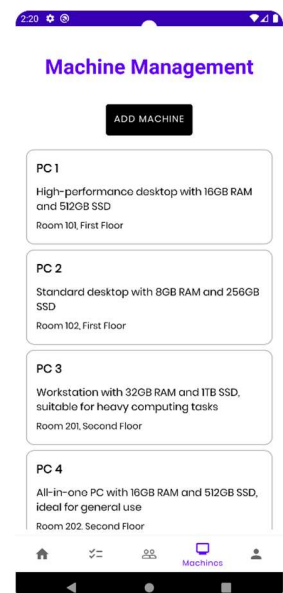


Figure 65. Admin Machine Management Page

2. Adding New Machines

- **Description:** Administrators can add new machines through the CreateMachineActivity, which provides a form for inputting machine details.
- **Implementation:** The CreateMachineActivity includes input fields for machine name, description, and location. Upon submission, the machine details are saved to Firebase Realtime Database.

```
class CreateMachineActivity : AppCompatActivity() {  
  
    private lateinit var etName: EditText  
    private lateinit var etDescription: EditText  
    private lateinit var etLocation: EditText  
    private lateinit var btnSave: Button  
    @ Cesar Vieira *  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_create_machine)  
        etName = findViewById(R.id.etMachineName)  
        etDescription = findViewById(R.id.etMachineDescription)  
        etLocation = findViewById(R.id.etMachineLocation)  
        btnSave = findViewById(R.id.btnSaveMachine)  
        btnSave.setOnClickListener { it View! }  
            saveMachine()  
        }  
    }  
    @ Cesar Vieira *  
    private fun saveMachine() {  
        val name = etName.text.toString().trim()  
        val description = etDescription.text.toString().trim()  
        val location = etLocation.text.toString().trim()  
  
        if (name.isNotEmpty() && description.isNotEmpty() && location.isNotEmpty()) {  
            val machineId = FirebaseDatabase.getInstance().reference.child( pathString: "pcs").push().key  
            val machine = Machine(name, description, location)  
            if (machineId != null) {  
                FirebaseDatabase.getInstance().reference.child( pathString: "pcs").child(machineId) DatabaseReference  
                    .setValue(machine) Task<Void>  
                    .addOnSuccessListener { it:Void!  
            }  
        }  
    }  
}
```

Figure 67. CreateMachineActivity

2:34

Create Machine

Machine Name

Description

Location

SAVE

Figure 66. Create Machine Page

3. Updating Machine Details

- **Description:** Administrators can update machine details using the UpdateMachineActivity. This activity displays current machine details and allows modifications.
- **Implementation:** The UpdateMachineActivity receives machine details via Intent extras and updates the machine entry in Firebase Realtime Database upon submission.

```
@ Cesar Vieira  
private fun updateMachine() {  
    val newName = editName.text.toString()  
    val newDescription = editDescription.text.toString()  
    val newLocation = editLocation.text.toString()  
  
    val database = FirebaseDatabase.getInstance().getReference( path: "pcs")  
    val machineUpdate = mapOf(  
        "name" to newName,  
        "description" to newDescription,  
        "location" to newLocation  
    )  
  
    database.child(machineId).updateChildren(machineUpdate).addOnCompleteListener { task ->  
        if (task.isSuccessful) {  
            Toast.makeText( context: this, text: "Machine updated successfully", Toast.LENGTH_SHORT).show()  
            finish() // Close the activity  
        } else {  
            Toast.makeText(  
                context: this,  
                text: "Failed to update machine: ${task.exception?.message}",  
                Toast.LENGTH_LONG  
            ).show()  
        }  
    }  
}
```

Figure 69. UpdateMachineActivity updateMachine function

2:36

Update Machine

PC 1

High-performance desktop with 16GB RAM and 512GB SSD

Room 101, First Floor

UPDATE

DELETE MACHINE

Figure 68. Update Machine Page

4. Deleting Machines

- **Description:** Administrators can delete machines from the UpdateMachineActivity. A delete button allows administrators to remove the machine entry from Firebase.
- **Implementation:** The deleteMachine method in UpdateMachineActivity removes the machine entry from Firebase and confirms the deletion with a Toast message.

```
private fun deleteMachine() {  
    val database = FirebaseDatabase.getInstance().getReference( path: "pcs")  
  
    database.child(machineId).removeValue().addOnCompleteListener { task ->  
        if (task.isSuccessful) {  
            Toast.makeText( context: this, text: "Machine deleted successfully", Toast.LENGTH_SHORT).show()  
            finish() // Close the activity  
        } else {  
            Toast.makeText(  
                context: this,  
                text: "Failed to delete machine: ${task.exception?.message}",  
                Toast.LENGTH_LONG  
            ).show()  
        }  
    }  
}
```

Figure 70. UpdateMachineActivity deleteMachine function

Behind the Scenes

Real-Time Machine Management

- **Description:** Machine data is managed using Firebase Realtime Database, ensuring that all changes are reflected in real-time.
- **Implementation:** The AdminMachinesViewModel listens for real-time updates on machine data and dynamically updates the UI, ensuring administrators always see the latest information.

```
class AdminMachinesViewModel : ViewModel() {  
  
    private val _machines = MutableLiveData<List<Machine>>()  
    val machines: LiveData<List<Machine>> = _machines  
  
    @ Cesar Vieira  
    init {  
        fetchMachines()  
    }  
  
    @ Cesar Vieira  
    private fun fetchMachines() {  
        val database = FirebaseDatabase.getInstance().getReference( path: "pcs")  
        database.addValueEventListener(object : ValueEventListener {  
            override fun onDataChange(snapshot: DataSnapshot) {  
                val machinesList = mutableListOf<Machine>()  
                for (machineSnapshot in snapshot.children) {  
                    val machine = machineSnapshot.getValue(Machine::class.java)  
                    if (machine != null) {  
                        // Include ID in the machine object  
                        machinesList.add(machine.copy(id = machineSnapshot.key!!))  
                    }  
                }  
                _machines.value = machinesList  
            }  
        })  
  
        override fun onCancelled(error: DatabaseError) {  
            // Handle database error  
        }  
    }  
}
```

Figure 71. AdminMachinesViewModel

Rationale

The Admin Manage Machines component addresses two critical needs: effective administrative control and a seamless user experience.

User Experience

- **User-Friendly Navigation:** All in all, AdminMachinesFragment gives navigation that is easy to use with the machines. At every point, there are options to add, update, or delete the machine.
- **Detailed Forms:** both CreateMachineActivity and UpdateMachineActivity are equipped with a detailed machine information form. This ensures that information regarding the machines is filled out comprehensively and modified accordingly.
- **Real-Time Feedback:** The integration will keep the machine data updated in real-time. This will let administrators work with the most current information.

Technical Efficiency

- **Separation of Concerns:** The use of distinct classes for data handling (AdminMachinesViewModel), UI display (MachineAdapter), and user interaction (AdminMachinesFragment, CreateMachineActivity, and UpdateMachineActivity) promotes a clean architecture and maintainability.
- **LiveData and ViewModel:** Utilizing Android's LiveData and ViewModel architecture components facilitates responsive UI updates and lifecycle-aware data management, ensuring efficient data handling and user interactions.
- **Firebase Integration:** Using Firebase Realtime Database for managing machine data ensures data consistency and scalability. Firebase's real-time synchronization capabilities make it an ideal choice for this component, providing reliable data management.

6. Development Lifecycle

This Android application for dynamic shared resources allocation has been developed using an Agile methodology tailored for solo development. This choice was due to the flexible nature and iteration of processes that adapt best to the needs of the project and constraints of individual development.

Methodology

As reported at the initiative of the project, with the Agile methodology, there was an enforcement of the personal Scrum. In practice, this implied a single-developer-oriented version of the Scrum, where the traditional Scrum events such as Sprint Planning and Daily Stand-ups were simplified in an individualized manner. Personal agile method using eight lists helped me explicitly define and control my goals and actions to manage the project better:

1. Goals List
2. To-Do List
3. This Week's List
4. Today's List
5. Progress List
6. Hold List
7. Done List

This structured approach ensured that project objectives remained clear and progress continued to be tracked for every single stage through the project development lifecycle.

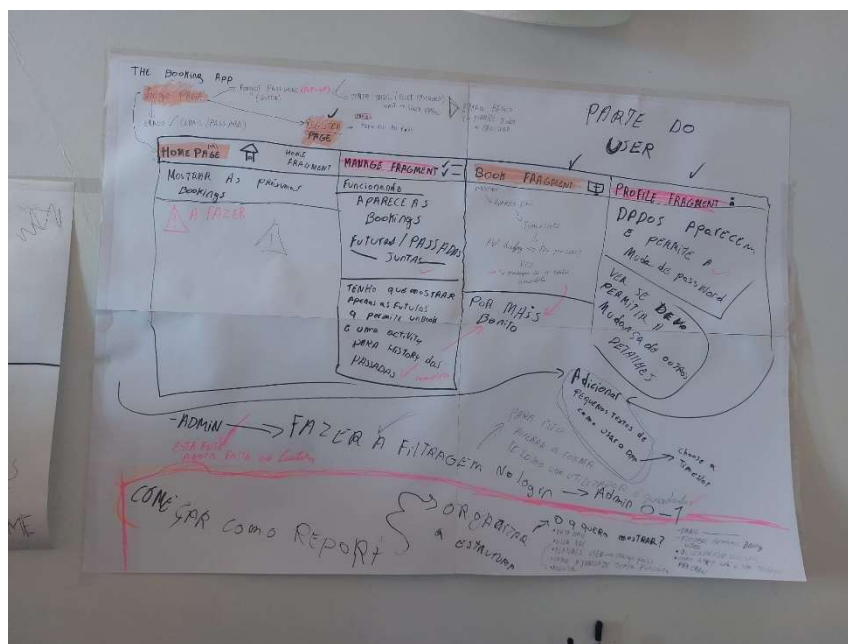


Figure 72. My To-do List

Stages of Development

Requirement Analysis

Requirements were gathered and defined through:

- Meetings with the supervisor, some teachers and colleagues to discuss and find the right features of the application.
- Analysis of similar booking systems
- Creation of use cases to capture user interactions

Validation: Requirements were reviewed with potential users and “stakeholders” (teachers) to ensure accuracy and completeness.

Design

The design phase included:

- Creation of wireframes and prototypes
- Development of system architecture
- UML diagrams (use case, sequence, and class diagrams)

Verification: Design documents were reviewed against requirements to ensure alignment.

Implementation

Implementation involved:

- Coding using Java and Kotlin for Android
- Version control with Git
- Continuous integration practices

Validation: Regular code reviews and adherence to coding standards ensured quality.

Testing

As mentioned in the first report, testing included:

- Manual Unit Tests: Meticulous assessments of individual components
- Direct Application Interaction: First-hand evaluation of usability and responsiveness

Verification: Test results were compared against expected outcomes to identify and resolve issues.

Validation and Verification

At each stage, different validation and verification processes had been put in place to make sure the application would meet all requirements and function correctly. It is one of the hallmarks of Agile methodologies: how continuous improvement and adaptation based on feedback and test results can be achieved in such a repetitive way. With this Agile-inspired development lifecycle, flexibility was retained, but systematic progress was guaranteed

towards creating a functional and user-friendly Android application for dynamic resource sharing.

7.Critical Analysis and Reflection

What Went Right/Wrong

Successes:

- The main goals of the project were achieved; a working application for booking shared computing resources was delivered.
- The use of Kotlin as the primary programming language proved beneficial. Its similarity to Java facilitated a smoother learning curve and development process.
- The implementation of the MVVM (Model-View-ViewModel) architecture, while initially challenging, led me to have a more organized and maintainable codebase.

Challenges:

- There was a significant learning curve associated with Firebase and the MVVM architecture. This initial struggle was demotivating but ultimately led to a deeper understanding of efficient app development practices.
- Android Studio, despite being the standard for Android development, presented several technical issues. Gradle build problems, crashes, and high CPU usage occasionally impeded progress and affected confidence.
- Some desired features, such as push notifications for booking cancellations, were not implemented due to Firebase's subscription-based pricing model for advanced features.

Improvements

For future projects, several improvements could be considered:

1. **Earlier adoption of architecture patterns:** Implementing MVVM from the project's outset could streamline development and reduce refactoring.
2. **More comprehensive technology research:** A deeper investigation into Firebase's pricing model and feature limitations could have informed better decision-making regarding app features.
3. **Alternative IDE exploration:** While Android Studio is the primary tool for Android development, exploring alternatives or optimizing its performance could mitigate some of the technical issues encountered.

Appraisal of the Product

The final product successfully meets the primary objectives established at the start of the project. It offers a functional platform for booking shared computing resources, which increases resource allocation efficiency. The user interface is intuitive and follows the Human-Computer Interaction principles discussed in the literature review. However, resource constraints prevented the implementation of some advanced features, such as push notifications.

Analysis of Approach

The use of an Agile methodology, specifically a personal Scrum approach, proved successful for this solo development project. It enabled iterative development and frequent reassessment of priorities. Despite the initial learning curve, the MVVM architecture allowed for a clear separation of concerns within the application, which improved maintainability and testability.

Tools and Technologies

Android Studio: While it presented some performance issues, Android Studio remained an invaluable tool for Android app development, providing a comprehensive IDE with powerful features for coding, debugging, and testing.

Kotlin: The decision to use Kotlin over Java was beneficial. Its concise syntax and null safety features contributed to more robust and readable code.

Firebase: For the booking system, Firebase offered a reliable backend solution with real-time database capabilities. Its tiered pricing structure, however, made it difficult to implement some sophisticated features.

MVVM Architecture: The Model-View-ViewModel architecture pattern, while challenging to learn, ultimately improved code organization and facilitated easier testing and maintenance of the application.

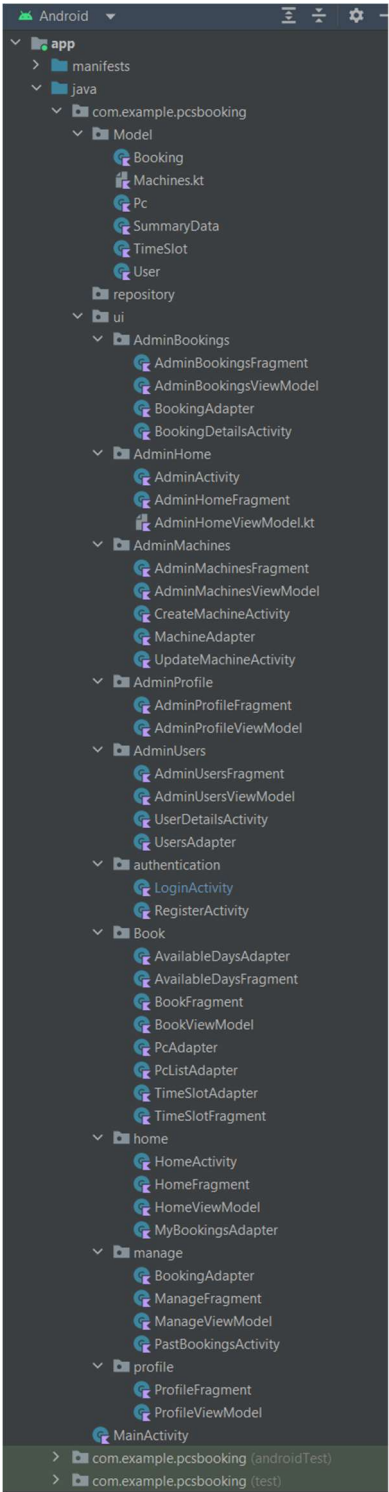
Although the project was completed successfully, problems arose on multiple levels, including a steep learning curve and technical difficulties. Experience gained through the implementation of the MVVM architecture, combined with ongoing Firebase learning, will be invaluable for future development projects. Despite not implementing all features as originally planned, it has met its core objectives and, with a foundation, future enhancements may be applied.

8. Reference List:

- Oluwaseun, B.R., Muyiwa, O., Olanrewaju, B.A., Omolaran, B.B. & Iyabo, B.S. 2016, 'StudentTeacher Online Booking Appointment System in Academic Institutions', Journal of Computer Science and Control Systems, vol. 9, no. 2, pp. 15-18.
- Haan, K. (2023) Remote work statistics and trends in 2023, Forbes. Available at: <https://www.forbes.com/advisor/business/remote-work-statistics/> (Accessed: 24 January 2024).
- Wrycza, S. and Maślankowski, J. (2020) 'Social media users' opinions on remote work during the COVID-19 pandemic. thematic and sentiment analysis', Information Systems Management, 37(4), pp. 288–297. doi:10.1080/10580530.2020.1820631.
- HP. (2022). Hybrid Work Report 2022: The Strategic Role of Digital Workspaces. Available at: <https://h20195.www2.hp.com/v2/GetDocument.aspx?docname=4AA8-2068ENW> (Accessed: 24 January 2024).
- HP Inc.. (2022). Does your organization utilize remote desktops?. Statista. Statista Inc.. Accessed: January 23, 2024. <https://www.statista.com/statistics/1368456/global-remote-desktopusage/>
- GeeksforGeeks (2022) *MVVM (Model View Viewmodel) architecture pattern in Android*, GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/mvvm-model-view-viewmodel-architecture-pattern-in-android/> (Accessed: 02 June 2024).
- GeeksforGeeks (2024) *Types of software architecture patterns*, GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/types-of-software-architecture-patterns/> (Accessed: 02 July 2024).
- Harwani, P. (2023) *MVVM (Model View Viewmodel) architecture pattern in Android*, Scaler Topics. Available at: <https://www.scaler.com/topics/android/mvvm-architecture-android/> (Accessed: 02 July 2024).
- *MVVM: Model-view-viewmodel architecture: Ramotion branding agency* (2024) *Web Design, UI/UX, Branding, and App Development Blog*. Available at: <https://www.ramotion.com/blog/what-is-mvvm/> (Accessed: 02 July 2024).

9.Appendices

Appendix A: Project Structure



Appendix B: UI Design

Normal User

